Unit 2

BASIC TYPES OF DATA IN MACHINE LEARNING

- A **data set is** a collection of related information or records.
- The information may be on **some entity or some subject area**.

BASIC TYPES OF DATA IN MACHINE LEARNING

- For example (Fig. 2.2), we may have a **data set on students** in which each record consists of information about a specific student.

- Again, we can have a **data set on student performance** which has records providing performance, i.e. marks on the individual subjects.

Student data set:

Roll Number	Name	Gender	Age
129/011	Mihir Karmarkar	M	14
129/012	Geeta Iyer	F	15
129/013	Chanda Bose	F	14
129/014	Sreenu Subramanian	M	14
129/015	Pallav Gupta	M	16
129/016	Gajanan Sharma	M	15

Student performance data set:

Roll Number	Maths	Science	Percentage
129/011	89	45	89.33%
129/012	89	47	90.67%
129/013	68	29	64.67%
129/014	83	38	80.67%
129/015	57	23	53.33%
129/016	78	35	75.33%

- FIG. 2.2 Examples of data set

Contd..

- **Each row of a data set** is called a record.

- Each data set also has multiple attributes, each of which gives **information on a specific characteristic.**

- For example, in the data set on students, there are **four attributes namely** Roll Number, Name, Gender, and Age, each of which understandably is a specific characteristic about the student entity.

Student data set:

Roll Number	Name	Gender	Age
129/011	Mihir Karmarkar	M	14
129/012	Geeta Iyer	F	15
129/013	Chanda Bose	F	14
129/014	Sreenu Subramanian	M	14
129/015	Pallav Gupta	M	16
129/016	Gajanan Sharma	M	15

Contd..

- **Attributes** can also be termed as feature, variable, dimension or field.

- Both the datasets**, Student and Student Performance**, are having **four features or dimensions;** hence they are told to have **four-dimensional data space.**

- **A row or record represents a point in the four-dimensional data space** as each row has specific values for **each of the four attributes or features**.

- **Value of an attribute,** may vary from record to record.

- For example, if we refer to the **first two records in the Student data set,** the value of attributes Name, Gender, and Age are different (Fig. 2.3).

Roll Number	Name	Gender	Age
129/011	Mihir Karmarkar	M	14
129/012	Geeta Iyer	F	15

FIG. 2.3 Data set records and attributes

Contd..

- Data can broadly be divided into following two types:
    1. Qualitative data
    2. Quantitative data

- **Qualitative data** provides **information** about the quality of an object or information which **cannot be measured.**

- For example, if we consider the **quality of performance of students** in terms of 'Good', 'Average', and 'Poor', it falls under the **category of qualitative data.**

- Also, **name or roll number of students** are information that cannot be measured using some scale of measurement.
- So they would fall under qualitative data.

- **Qualitative data** is also called **categorical data**.

Contd..

- **Qualitative data can be further subdivided** into two types as follows:
    1. Nominal data
    2. Ordinal data

- **Nominal data** is one which has no numeric value, but a **named value.**

- It is used for assigning named values to attributes.

- Nominal values cannot be quantified.

- Examples of nominal data are
    1. Blood group: A, B, O, AB, etc.
    2. Nationality: Indian, American, British, etc.
    3. Gender: Male, Female, Other

Contd..

- It is obvious, mathematical operations such as addition, subtraction, multiplication, etc. **cannot be performed** on nominal data.

- Statistical functions such as mean, variance, etc. can also **not be applied on nominal data.**

- A basic count is possible.

- So mode, i.e. most frequently occurring value, can be identified for nominal data.

- **Note :-**
- A **special case of nominal data is** when only two labels are possible, e.g. pass/fail as a **result of an examination**.

- This **sub-type of nominal data** is called 'dichotomous'.

Contd..

- **Ordinal data**, in addition to possessing the properties of nominal data, can also be **naturally ordered.**

- This means ordinal data also assigns named values to attributes but unlike nominal data, they can be **arranged in a sequence of increasing or decreasing value** so that we can say whether a **value is better than or greater than another value.**

- Examples of ordinal data are
    1. Customer satisfaction: ‘Very Happy’, ‘Happy’, ‘Unhappy’, etc.
    2. Grades: A, B, C, etc.
    3. Hardness of Metal: ‘Very Hard’, ‘Hard’, ‘Soft’, etc.

Contd..

- Like nominal data, basic counting is possible for ordinal data.
- Hence, the mode can be identified.
- Since ordering is possible in case of ordinal data, median, and quartiles can be identified in addition.
- Mean can still not be calculated.

Contd..

- **Quantitative data** relates to information about the quantity of an object – hence it can be measured.

- For example, if we consider the attribute 'marks', it can be **measured using a scale of measurement.**

- **Quantitative data** is also termed as numeric data.

- There are two types of quantitative data:
  1. Interval data
  2. Ratio data

Contd..

- **Interval data** is **numeric data** for which not only the order is known, but the exact difference between values is also known.

- An ideal **example** of interval data is Celsius temperature.

- The difference between each value remains the same in Celsius temperature.

- For example, the **difference between 12°C and 18°C degrees is measurable** and is 6°C as in the case of difference between 15.5°C and 21.5°C.

- Other examples include date, time, etc.

Contd..

- **For interval data**, mathematical operations such as addition and subtraction are possible.

- For that reason, for interval data, the **central tendency can** be measured by mean, median, or mode.

- **Standard deviation** can also be calculated.

- However, interval data do not have something called a '**true zero' value.**

- **For example**, there is nothing called '0 temperature' or 'no temperature'.
- Hence, only addition and subtraction applies for interval data.

- The **ratio cannot be applied.**

- This means, **we can say** a temperature of 40°C is equal to the temperature of 20°C + temperature of 20°C.

- However, we **cannot** say the temperature of 40°C means it is twice as hot as in temperature of 20°C.

Contd..

- **Ratio data** represents numeric data for which **exact value can be measured**.

- **Absolute zero** is **available for ratio data.**

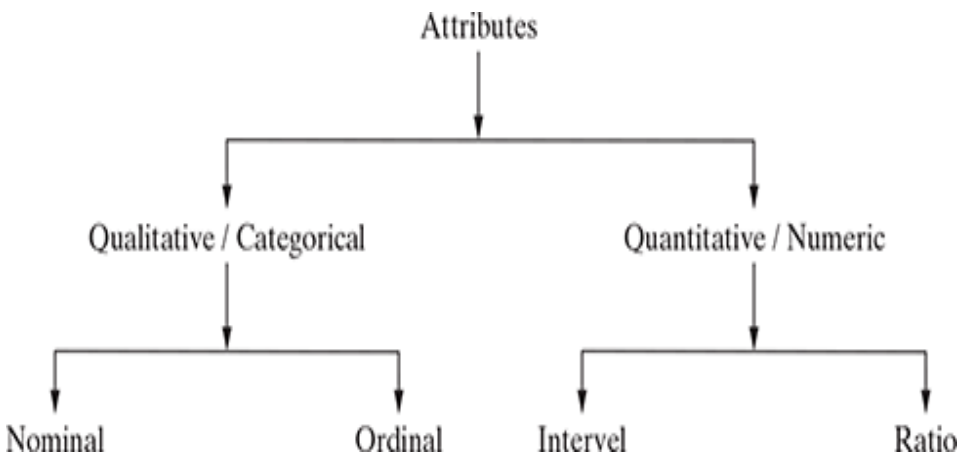
- Also, **these variables** can be added, subtracted, multiplied, or divided.

- The central tendency can be **measured by mean, median, or mode and methods of dispersion** such as **standard deviation**.

- Examples of **ratio data** include height, weight, age, salary, etc.

Contd..

- Figure 2.4 gives a summarized view of different types of data that we may find in a typical machine learning problem.



- **FIG. 2.4** Types of data

- **Attributes** can also be categorized into types based on a number of values that can be assigned.
- **The attributes** can be either discrete or continuous based on this factor.

Contd..

- **Discrete attributes** can assume a finite or countably infinite number of values.

- **Nominal attributes** such as roll number, street number, pin code, etc. can have a **finite number of values** whereas numeric attributes such as count, rank of students, etc. can have **countably infinite values.**

- **A special type of discrete attribute** which can assume two values only is called binary attribute.

- **Examples of binary attribute i**nclude male/ female, positive/negative, yes/no, etc.

Contd..

- **Continuous attributes** can assume any possible value which is a **real number.**

- **Examples of continuous attribute** include length, height, weight, price, etc.

- **Note:**
- In general, **nominal and ordinal attributes** are discrete.

- On the other hand, **interval and ratio attributes** are conti nuous, barring a few exceptions, e.g. 'count' attribute.

EXPLORING STRUCTURE OF DATA

- In machine learning, the **two basic data types – numeric and categorical**.
- In a data set ,we need to understand which of the attributes are numeric and which are categorical in nature.
- This is because, the approach of **exploring numeric data is different than the approach of exploring categorical data**.
- In case of a **standard data set**, we may have the data dictionary available for reference.
- **Data dictionary** is a metadata repository, i.e. the repository of all information related to the structure of each data element contained in the data set.
- The **data dictionary** gives detailed information on each of the attributes – the **description** as well as the **data type and other relevant details.**
- In case the data dictionary is not available, we need to **use standard library function of the machine learning tool** that we are using and get the details.

This slide -- For information

- University of California, Irvine (UCI) Machine Learning Repository (http://archive.ics.uci.edu/ml/index.php) is a collection of 400+ data sets which serve as benchmarks for researchers and practitioners in the machine learning community.

Contd..

- The data set that taken for reference is the Auto MPG data set available in the UCI repository.
- Figure 2.5 is a snapshot of the **first few rows of the data set.**

mpg	cylinder	displace-ment	horse-power	weight	accel-eration	model year	origin	car name
18	8	307	130	3504	12	70	1	Chevrolet chev-elle malibu
15	8	350	165	3693	11.5	70	1	Buick skylark 320
18	8	318	150	3436	11	70	1	Plymouth satellite
16	8	304	150	3433	12	70	1	Amc rebel sst
17	8	302	140	3449	10.5	70	1	Ford torino
15	8	429	198	4341	10	70	1	Ford galaxie 500
14	8	454	220	4354	9	70	1	Chevrolet impala
14	8	440	215	4312	8.5	70	1	Plymouth fury iii
14	8	455	225	4425	10	70	1	Pontiac catalina
15	8	390	190	3850	8.5	70	1	Amc acbassador dpl
15	8	383	170	3563	10	70	1	Dodge challenger se
14	8	340	160	3609	8	70	1	Plymouth ' cuda 340
15	8	400	150	3761	9.5	70	1	Chevrolet monte carlo
14	8	455	225	3086	10	70	1	Buick estate wagon (sw)
24	4	113	95	2372	15	70	3	Toyota corona mark ii
22	6	198	95	2933	15.5	70	1	Plymouth duster
18	6	199	97	2774	15.5	70	1	Amc hornet

Contd..

- As is quite evident from the data, the **attributes such as‘mpg’, ‘cylinders’, ‘displacement’, ‘horsepower’, ‘weight’, ‘acceleration’, ‘model year’, and ‘origin’** are all numeric.

- Out of these attributes, ‘**cylinders’, ‘model year’, and ‘origin**’ are discrete in nature as the only finite number of values can be assumed by these attributes.

- The remaining of the numeric attributes, i.e. ‘**mpg’, ‘displacement’, ‘horsepower’, ‘weight’, and ‘acceleration**’ can assume any real value.

- Hence, these attributes are continuous in nature.

Contd..

- The only **remaining attribute 'car name'** is of type categorical, or more specifically nominal.

- This **data set is regarding prediction of fuel consumption in miles per gallon,** i.e. the numeric attribute 'mpg' is the target attribute.

- With this understanding of the data set attributes, we can start exploring the numeric and categorical attributes separately.

  - 1 Exploring numerical data
  - 2 Plotting and exploring numerical data
  - 3 Exploring categorical data
  - 4 Exploring relationship between variables

Exploring numerical data

- There are **two most effective mathematical plots** to explore numerical data – box plot and histogram.

1. Understanding central tendency

- To understand the **nature of numeric variables**, we can apply the measures of central tendency of data, i.e. mean and median.
- **In statistics, measures of central tendency** help us understand the central point of a set of data.
- **Mean**, by definition, is a sum of all data values divided by the count of data elements.
- For example, mean of a set of observations – 21, 89, 34, 67, and 96 is calculated as below.

$$\text{Mean} = \frac{21 + 89 + 34 + 67 + 96}{5} = 61.4$$

- If the above **set of numbers represents marks of 5 students** in a class, the mean marks, or the falling in the middle of then range is 61.4.

Contd..

- **Mean**, by definition, is a sum of all data values divided by the count of data elements.

- For example, mean of a set of observations – 21, 89, 34, 67, and 96 is calculated as below.

- If the above **set of numbers represents marks of 5 students** in a class, the mean marks, or the falling in the middle of then range is 61.4.

$$\text{Mean} = \frac{21 + 89 + 34 + 67 + 96}{5} = 61.4$$

Contd..

- **Median**, is the **value** of the element **appearing in the middle of an ordered list of data elements.**

- If we consider the **above 5 data elements,** the ordered list would be –

  21, 34, 67, 89, and 96.

- Since there are 5 data elements, **the 3rd element in the ordered list is considered as the median.**

- Hence, the **median value of this set of data is 67.**

Contd..

- There might be a natural curiosity to understand **why two measures of central tendency are reviewed.**

- The reason is **mean and median are impacted differently by data values** appearing at the beginning or at the end of the range.

- **Mean** being **calculated from the cumulative sum of data values**, is impacted if too many data elements are having values closer to the far end of the range, i.e. close to the maximum or minimum values.

- It is especially **sensitive to outliers, i.e. the values which are unusually high or low, compared to the other values.**

- **Mean** is likely to get shifted drastically even due to the presence of a small number of outliers.

- If we observe that **for certain attributes the deviation between values of mean and median are quite high**, we should investigate those attributes further and try to **find out the root cause along with the need for remediation.**

Contd..

- So, in the **context of the Auto MPG data set**, let's try to **find out** for each of the **numeric attributes the values of mean and median.**
- We can also find out if the deviation between these values is large.
- In Figure 2.6, the **comparison between mean and median** for all the attributes has been shown.
- We can see that for the attributes such as 'mpg', 'weight', 'acceleration', and 'model.year' the deviation between mean and median is not significant which means the **chance of these attributes having too many outlier values is less.**
- However, the deviation is **significant for the attributes 'cylinders', 'displacement' and 'origin'.**
- So, we **need to further drill down and look at some more statistics** for these attributes.

FIG. 2.6 Mean vs. Median for Auto MPG

	mpg	cylin-ders	dis-place-ment	horse-power	weight	accel-eration	model year	origin
Median	23	4	148.5	?	2804	15.5	76	1
Mean	23.51	5.455	193.4	?	2970	15.57	76.01	1.573
Deviation	2.17	26.67%	23.22%		5.59%	0.45%	0.01%	36.43%
	Low	High	High		Low	Low	Low	High

Contd..

- Also, **there is some problem** in the values of the attribute 'horsepower' because of which the **mean and median calculation is not possible.**

- With a bit of investigation, we can find out that the problem is occurring because of the 6 data elements, as shown in Figure 2.7, **do not have value for the attribute 'horsepower'.**

mpg	cylinders	displace-ment	horse-power	weight	accel-eration	model year	origin	car name
25	4	98	?	2046	19	71	1	Ford pinto
21	6	200	?	2875	17	74	1	Ford maverick
40.9	4	85	?	1835	17.3	80	2	Renault lecar deluxe
23.6	4	140	?	2905	14.3	80	1	Ford mustang cobra
34.5	4	100	?	2320	15.8	81	2	Renault 18i
23	4	151	?	3035	20.5	82	1	Amc concord di

- **FIG. 2.7** Missing values of attribute 'horsepower' in Auto MPG

Contd..

- For that reason, **the attribute 'horsepower' is not treated as a numeric.**

- That's **why the operations applicable on numeric variables**, like mean or median, are failing.

- So we have to **first remediate the missing values** of the attribute 'horsepower' before being **able to do any kind of exploration.**

2. Understanding data spread

- Now that we have explored the central tendency of the **different numeric attributes**, we have a **clear idea of which attributes have a large deviation between mean and median.**

- Let's look closely at those attributes.
- To drill down more, we need to lo**ok at the entire range of values of the attributes**, though not at the level of data elements as that may be too vast to review manually.

- So we will take a granular view of the data spread in the form of
    1. Dispersion of data
    2. Position of the different data values

Contd..

- ***Measuring data dispersion***
- Consider the data values of two attributes

  1. Attribute 1 values : 44, 46, 48, 45, and 47
  2. Attribute 2 values : 34, 46, 59, 39, and 52

- Both the **set of values have a mean and median of 46**.

- However, the first set of values that is of attribute 1 is more concentrated or clustered around the mean/median value whereas the **second set of values of attribute 2 is quite spread out or dispersed.**

- To measure the extent of dispersion of a data, or to **find out how much the different values of a data are spread out**, the variance of the data is measured.

Contd..

- where *x* is the variable or attribute whose **variance is to be measured** and *n* is the number of observations or values of variable x.

$$\text{Variance}_{(x)} = \frac{\sum_{i=1}^{n} x_i^2}{n} - \left(\frac{\sum_{i=1}^{n} x_i}{n}\right)^2$$, where x is the variable or

- Standard deviation of a data is measured as follows:

$$\text{Standard deviation}\ (x) = \sqrt{\text{Variance}\ (x)}$$

Contd..

- Larger value of variance or standard deviation indicates **more dispersion in the data and vice versa.**

- In the above example, **let's calculate the variance of attribute 1 and that of attribute 2.**

- For attribute 1,

$$\text{Variance} = \frac{\sum_{i=1}^{n} x_i^2}{n} - \left(\frac{\sum_{i=1}^{n} x_i}{n}\right)^2$$

$$= \frac{44^2 + 46^2 + 48^2 + 45^2 + 47^2}{5} - \left(\frac{44 + 46 + 48 + 45 + 47}{5}\right)^2$$

$$= \frac{1936 + 2116 + 2304 + 2025 + 2209}{5} - \left(\frac{230}{5}\right)^2 = \frac{10590}{5} - (46)^2 = 2$$

Contd..

For attribute 2,

$$\text{Variance} = \frac{\sum_{i=1}^{n} x_i^2}{n} - \left(\frac{\sum_{i=1}^{n} x_i}{n}\right)^2$$

$$= \frac{34^2 + 46^2 + 59^2 + 39^2 + 52^2}{5} - \left(\frac{34 + 46 + 59 + 39 + 52}{5}\right)^2$$

$$= \frac{1156 + 2116 + 3481 + 1521 + 2704}{5} - \left(\frac{230}{5}\right)^2 = \frac{10978}{5} - (46)^2 = 79.6$$

- So it **is quite clear** from the measure that attribute 1 values are quite concentrated around the mean while **attribute 2 values are extremely spread out.**

- Since **this data was small,** a visual inspection and understanding were possible and that **matches with the measured value.**

IDE used for Machine Learning

- "Different tools are used in the data science and machine learning (ML) ecosystem, and they serve different purposes,
  - such as Anaconda, Google Colab, and Jupyter Notebooks."

- **Jupyter Notebook** is an **open-source web application** for interactive coding.

- **Anaconda** is a d**istribution that simplifies the installation and package management** for data science tools (including Jupyter),

- **Google Colab** is a **cloud-hosted, managed version of Jupyter Notebooks** offering free access to computational resources like GPUs.

IDE used for Machine Learning

- **JupyterLab** is the latest web-based interactive development environment for **notebooks, code, and data.**

- It allows users to combine live code, narrative text, equations, and visualizations in a single document.

- It's the base technology for interactive data science documents.

- Its **flexible interface** allows users to configure and arrange workflows in data science, scientific computing, computational journalism, and machine learning.

- A modular design invites extensions **to expand and enrich functionality**. (https://jupyter.org/)

IDE used for Machine Learning

- **Google Colab** is essentially a **hosted version of a Jupyter Notebook** that runs on Google's cloud infrastructure.

- It removes the need for installation and setup, provides **integrated access to free computational power (GPUs/TPUs)**, and facilitates easy sharing and collaboration, much like Google Docs.

- Colab is especially well suited to machine learning, data science, and education. (https://colab.google/)
- (GPUs (Graphics Processing Units) and TPUs (Tensor Processing Units) are specialized, high-performance hardware accelerators for AI, with GPUs offering versatile, general-purpose parallel computing (ideal for diverse frameworks like PyTorch/TensorFlow) and TPUs providing specialized, high-efficiency ASIC performance tailored for large-scale, dense matrix operations within Google Cloud.)

IDE used for Machine Learning

- **Anaconda** is a popular, open-source distribution of Python and R programming languages designed for **data science, machine learning, and AI.**

- It simplifies package management and deployment, featuring over 250 pre-installed data science libraries (like NumPy, pandas, scikit-learn, etc.) and the {Link: conda https://docs.conda.io/} package manager.

- It simplifies the process of setting up a complex data science environment on a **local machine, often including Jupyter Notebook** within its installation.

- It is widely used by data scientists for managing complex dependencies in environments.

Feature	Anaconda	Google Colab	Jupyter Notebook
Type	Python/R distribution & package manager	Cloud-based hosted service	Open-source web application
Environment	Local machine (requires installation)	Cloud (browser-based, no setup)	Local machine (requires installation/setup)
Resources	Uses local computer's CPU/GPU/memory	Provides free access to GPUs/TPUs (with usage limits)	Uses local computer's CPU/GPU/memory
Collaboration	Local files can be shared manually or via source control	Seamless collaboration and sharing via Google Drive	Can export/share documents, but lacks real-time collaboration features like Colab
Package Management	Uses Conda and Pip for easy package /environment management	Comes with many pre-installed libraries, fewer customization options for system packages	Uses Pip or Conda for package management, depends on installation method
Ideal For	Full control over the environment, large-scale processing, offline work	Beginners, collaborative projects, deep learning requiring GPUs, easy setup	Interactive exploration, teaching, testing ideas, running code block by block

Python Libraries for Machine Learning

- **ML Environment Setup** -- Anaconda, Google Colab, Jupyter Notebooks

- **Python scripts for ML (.ipynb / .py)--**
- Python scripts for machine learning are typically implemented as **Jupyter Notebooks (.ipynb)** for interactive development and experimentation, or as plain **Python scripts (.py)** for production and automation.

- **Both formats suppor**t all major ML libraries like scikit-learn, TensorFlow, and PyTorch.
- Python Libraries used for Machine Learning
  - **NumPy**
  - **Pandas**
  - **Matplotlib**
  - **Scikit-learn**

NumPy Tutorial - Python Library

- **NumPy** is a core Python library for **numerical computing**, built for **handling large arrays and matrices efficiently**.

- It is significantly **faster than Python's built-in lists** because it uses optimized C language style storage where actual values are stored at contiguous locations (not object reference).

- It also **supports vectorized computations. (**which perform operations on entire arrays at once, rather **than iterating through individual elements using explicit Python loop**s**)**

What is NumPy Used for?

- With NumPy, you can perform a wide range of numerical operations, including:
  - **Creating and manipulating arrays.**
  - Performing element-wise and matrix operations.
  - Generating **random numbers and statistical calculations.**
  - Conducting linear algebra operations.
  - Working with **Fourier transformations.**
  - Handling missing values efficiently in datasets.

Why Learn NumPy?

- NumPy **speeds up math operations** like addition and multiplication on large groups of numbers compared to regular Python.

- It's **good for handling large lists of numbers (arrays),** so you don't have to write complicated loops.

- It gives **ready-to-use functions** for statistics, algebra and random numbers.

- Libraries like Pandas, SciPy, TensorFlow and many others **are built on top of NumPy.**

- NumPy uses less memory and stores data more efficiently, which matters when working with lots of data.

Key Features of NumPy

- NumPy has various features that make it popular over lists.
- **N-Dimensional Arrays**: NumPy's core feature is ndarray, a N-dimensional array object that **supports homogeneous data types.**

- **Arrays with High Performance**: Arrays are stored in **contiguous memory locations, enabling faster computations** than Python lists (Please see Numpy Array vs Python List for details).

- **Broadcasting:** This allows element-wise computations between **arrays of different shapes.** It **simplifies operations on arrays of various** shapes by automatically aligning their dimensions without creating new data.

- **Vectorization: Eliminates the need for explicit Python loops** by applying operations directly on entire arrays.

- **Linear algebra**: NumPy contains **routines for linear algebra operations**, such as matrix multiplication, decompositions, and determinants.

Installing NumPy in Python

- To begin using NumPy, you need **to install it first.**

- This can be done using the following pip command:
  - **pip install numpy**

- Once installed, import the library with the alias np
  - **import numpy as np**

Basics of NumPy Arrays

- NumPy stands for Numerical Python and is used for **handling large, multi-dimensional arrays and matrices.**

- Unlike Python's built-in lists NumPy arrays provide **efficient storage and faster processing for numerical and scientific computations**.

- It offers functions for **linear algebra and random number generation** making it important for **data science and machine learning.**

Types of Array

- Various types of arrays are as follows:
- **1. One-Dimensional Array**
  - A one-dimensional array is a **type of linear array.**

1	2	3	4	5

- **2. Multi-Dimensional Array**
- A Multi-Dimensional Array is an array that **can store data in more than one dimension such as rows and columns**.
- In simple terms it is an array of arrays.
- For example, a 2D array is like a table with rows and columns where each element is accessed by two indices: one for the row and one for the column.
- Higher dimensions like **3D arrays involve adding additional layers.**

1	2	3	4	5
6	7	8	9	10

One-Dimensional Array

- **Example:**

```
import numpy as np
a = [1, 2, 3, 4]
arr = np.array(a)
print("List in python : ", a)
print("Numpy Array in python :",
    arr)
```

Output:

```
List in python :  [1, 2, 3, 4]
Numpy Array in python : [1 2 3 4]
```

- **Check data type for list and array:**

```
print(type(list_1))
print(type(sample_array))
```

Output:

```
<class 'list'>
<class 'numpy.ndarray'>
```

Multi-Dimensional Array

- **Example:**

```
import numpy as np
list_1 = [1, 2, 3, 4]
list_2 = [5, 6, 7, 8]
list_3 = [9, 10, 11, 12]
sample_array = np.array([list_1, list_2, list_3])
print("Numpy multi dimensional array in python\n", sample_array)
```

- **Output**

```
Numpy multi dimensional array in python
[[ 1 2 3 4]
[ 5 6 7 8]
 [ 9 10 11 12]]
```

Note: use [ ] operators inside numpy.array() for multi-dimensional

Creating NumPy Arrays

- **1. Using np.array:**
- Use np.array() when you want to **convert Python lists** into NumPy arrays.

- **numpy.array()**: Numpy **array object in Numpy is called ndarray.**

- We can create ndarray using this function.

- **Syntax:** numpy.array(parameter)

- **Example:**

```
import numpy as np
a1 = np.array([1, 2, 3])  # 1D array
a2 = np.array([[1, 2], [3, 4]])  # 2D array
a3 = np.array([[[1, 2], [3, 4]],  # 3D  array
               [[5, 6], [7, 8]]])

print(a1)
print(a2)
print(a3)
```

Output

```
[1 2 3]
 [[1 2]
 [3 4]]
 [[[1 2]
  [3 4]]
 [[5 6]
  [7 8]]]
```

Contd..

2. Using Numpy Functions: NumPy provides quick utility functions for creating arrays filled with **zeros, ones, or ranges**:

- **np.zeros((rows, cols))** – Creates an array of 0s.

- **np.ones((shape))** – Creates an array of 1s.

- **np.arange(start, stop, step, dtype)** – Similar to Python's range() but returns an array.
  - **start (optional):** The starting value of the sequence (inclusive). Defaults to 0.
  - **stop (required):** The end value of the sequence (exclusive, meaning it is not included in the array).
  - **step (optional):** The **spacing between consecutive values**. Defaults to 1.
  - **dtype (optional):** The **data type of the output array elements**. If not specified, the type is inferred from the **types of start, stop, and step**. Default data type is float.

Contd..

Example

```
import numpy as np

a0 = np.zeros((3, 3))
a1  = np.ones((2, 2))
ar  = np.arange(0, 10, 2)

print(a0)
print(a1)
print(ar)
```

Output

```
[[0. 0. 0.]
[0. 0. 0.]
 [0. 0. 0.]]
 [[1. 1.]
[1. 1.]]
[0 2 4 6 8]
```

NumPy Array Indexing

- **Array indexing** in NumPy refers to the method of **accessing specific elements or subsets of data within an array.**

- This feature allows us **to retrieve, modify and manipulate data at specific positions or ranges** helps in making it easier to work with large datasets.

- **1. Accessing Elements in 1D Arrays**
- A 1D NumPy array is a **sequence of values with positions called indices which starts at 0.**

- We access elements by using **these indices in square brackets like arr[0] for the first element.**

- Negative indices count from **the end so arr[-1] gives the last element.**

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50])
print(arr[0])
```

- **Output:** 10

- **2. Accessing Elements in Multidimensional Arrays**
- In this we will see how to access **elements in both 2D and 3D arrays using specific indices.**
- **2D Arrays**: We can access elements by **specifying both row and column** indices like **matrix[row, column]**.

```
import numpy as np
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(matrix[1, 2])
```

Output: 6

```
import numpy as np

    a1 = np.array([10, 20, 30, 40, 50])
    print(a1[2])      # single element
    print(a1[-1])     # last element

    a2 = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])

    print(a2[1, 0])   # row 1, column 0
```

Output

```
30
50
 4
```

Slicing Arrays

- It allows us to **extract a range of elements using** the format **start:stop:step**.
  - If we don't pass start its considered 0
  - If we don't pass end its considered length of array in that dimension
  - If we don't pass step its considered 1

- This can be done **for both 1D and multidimensional arrays** which allows us to select ranges of elements or submatrices easily.

Slicing Arrays

- **Slicing 1D Arrays:** For a 1D array, **slicing returns a subset of elemen**ts between the **start** and **stop** indices.

- **Example**

```
import numpy as np
arr = np.array([0, 1, 2, 3, 4, 5])
print(arr[1:4])
```

- **Output:**

```
[1 2 3]
```

- Here **arr[1:4]** slices the array starting at index 1 up to (but not including) index 4 so it returns the elements [1, 2, 3].

Contd..

- **Slicing Multidimensional Arrays:**
- In this slicing can be applied to **each dimension separately which allows us to extract submatrices or smaller blocks of data.**

```
import numpy as np
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(matrix[0:2, 1:3])
```

- **Output**

```
[[2 3]
 [5 6]]
```

Reshape NumPy Array - Python

- Reshaping means **changing the shape of an array.**
- The **shape of an array is** the number of elements in each dimension.
- By reshaping we can add or remove dimensions or change number of elements in each dimension.
- Reshaping in NumPy refers **to modifying the dimensions of an existing array without changing its data**.
- The reshape() function is used for this purpose.
- It reorganizes the elements into a new shape, which is useful in **machine learning, matrix operations and data preparation.**

Reshape NumPy Array - Python

- **Example 1:**
- This example **converts a 1-D array into a 2-D array** by specifying rows and columns that match the total number of elements.

```
import numpy as np
a = np.array([1, 2, 3, 4, 5, 6])
r = a.reshape(2, 3)
print(r)
```

- **Output**

```
[[1 2 3]
[4 5 6]]
```

- **Explanation:**
- a.reshape(2, 3) arranges the 6 elements into 2 rows and 3 columns, forming a 2-D matrix.

Contd..

Syntax

array.reshape(shape)

Parameter:

shape - Tuple, defining the new dimensions.

One dimension can be -1, letting NumPy auto-calculate it based on the total elements. Using -1 in a reshape command tells the computer to **automatically calculate** the size of that specific dimension based on the total number of elements and the other dimensions provided.

Returns: A new reshaped ndarray.

- **Example 2:**
- This example **creates a 3-D array** by grouping the original elements into blocks, each containing equal-sized 2-D sections.

```
import numpy as np
a = np.array([1, 2, 3, 4, 5, 6, 7, 8])
r = a.reshape(2, 2, 2)
print(r)
```

- **Output**
  - [[[1 2]
  - [3 4]]
  - [[5 6]
  - [7 8]]]

- **Explanation: a.reshape(2, 2, 2)** transforms the array **into 2 blocks, each containing a 2×2 matrix, forming a 3-D structure.**

- **Example Code (NumPy):**

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
new_arr = arr.reshape(3, -1)
print("The value of new_arr will be")
print(new_arr)
```

- **arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])**
  - This line creates a one-dimensional NumPy array **named arr containing integers from 1 to 12.**

- **new_arr = arr.reshape(3, -1)**:
- This is the core operation.
  - reshape(3, ...) instructs NumPy to **reorganize the 1D array into a new 2D array with 3 rows.**
  - The -1 is a wildcard. NumPy **automatically calculates the number of columns required to accommodate** all 12 elements (12 elements / 3 rows = 4 columns).

- **Result**
- After executing the corrected code, **the original 1D array is transformed into a 2D array with 3 rows and 4 columns:**
  - The value of new_arr will be:
  - [[1, 2, 3, 4],
  - [5, 6, 7, 8],
  - [9, 10, 11, 12]]

numpy.mean() in Python

- numpy.mean() is a NumPy function used to **calculate the average (arithmetic mean) of numeric values.**

- It can compute the **mean of a 1D list/array or compute mean row-wise** and **column-wise for multi-dimensional arrays**.

Example:

Input: *[1, 2, 3]*
Output: *2.0*

numpy.mean(arr, axis=None, dtype=None, out=None)

Parameters:

arr: Input array of numbers

axis: None - mean of all elements, **0** - column-wise mean and **1** - row-wise mean

dtype(Optional): type used while computing mean

out(Optional): array to store the result

Example 1: This example finds the average value of a 1D list using np.mean().

```
import numpy as np
arr = [20, 2, 7, 1, 34]
res = np.mean(arr)
print(res)
```

Output

```
12.8
```

Numpy numpy.resize() - Python

- The **numpy.resize() function** creates and returns a **new** array with the specified shape, without modifying the original array.

- It modifies the array permanently and adjusts its shape to the new dimensions.

- If the new shape **requires more elements than available**, NumPy **repeats the array elements.**

- If **less space is required, elements are truncated.**

- However, there is **an array *method*, ndarray.resize**(), which modifies the existing array **in-place**.

- **Syntax**

  numpy.resize(a, new_shape)

- **Parameters:**

  a: Input array to be resized.

  new_shape: Target shape (int or tuple).

  refcheck(optional): If True, checks whether the array is referenced elsewhere before resizing.

- This function creates and returns a **new array** with the specified new_shape. The **original array remains unchanged.**
- **Enlarging:** If the **new shape is larger**, the original array's elements are **repeated cyclically** to fill the extra space.
- **Shrinking:** If the new shape is smaller, the data is **truncated**.

```
import numpy as np

arr = np.array([1, 2, 3, 4])
# Resize to a larger shape (6 elements total)
new_arr_large = np.resize(arr, (6,))
print(f"Enlarged array: {new_arr_large}")

# Resize to a smaller shape (2 elements total)
new_arr_small = np.resize(arr, (2,))
print(f"Truncated array: {new_arr_small}")

# Resize a 2D array
arr_2d = np.array([[1, 2], [3, 4]])
new_arr_2d = np.resize(arr_2d, (3, 2))
print(f"Resized 2D array:\n{new_arr_2d}")
```

Output

```
Enlarged array: [1 2 3 4 1 2]
Truncated array: [1 2]
Resized 2D array:
[[1 2]
 [3 4]
 [1 2]]
```

Example:-

```
import numpy as np

original = np.array([1, 2, 3])

resized_func = np.resize(original, (4, 3))

print(f"Original array: {original}")

print(f"Resized array (function):\\n{resized_func}")
```

```
Original array: [1 2 3]
Resized array (function):
 [[1 2 3]
 [1 2 3]
 [1 2 3]
 [1 2 3]]
```

```
import numpy as np
a = np.array([[0,1],[2,3]])
print(np.resize(a,(2,3)))
print()
print(np.resize(a,(1,4)))
print()
print(np.resize(a,(2,4)))
print()
a.resize((3, 4))
print(a)
```

Output

```
[[0 1 2]
 [3 0 1]]

[[0 1 2 3]]

[[0 1 2 3]
 [0 1 2 3]]

[[0 1 2 3]
 [0 0 0 0]
 [0 0 0 0]]
```

- **Example 1:**

  This example resizes a **1D array of 6 elements into a 2×3 array**. No values need repetition or truncation.

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6])
arr.resize((2, 3))
print(arr)
```

```
Output
[[1 2 3]
 [4 5 6]]
```

Modified

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])

print("numpy.resize")
print(np.resize(arr,(4,3)),'\n')
arr.resize((4, 3))
print(arr)
```

```
Output
numpy.resize
[[1 2 3]
 [4 5 6]
 [1 2 3]
 [4 5 6]]

[[1 2 3]
 [4 5 6]
 [0 0 0]
 [0 0 0]]
```

- If sequence change–
- Same program of previous slide

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])

arr.resize((4, 3))
print(arr,'\n')

print("numpy.resize")
print(np.resize(arr,(4,3)),'\n')
```

Output

```
[[1 2 3]
 [4 5 6]
 [0 0 0]
 [0 0 0]]

numpy.resize
[[1 2 3]
 [4 5 6]
 [0 0 0]
 [0 0 0]]
```

- **Example 2:**
- This example **resizes a 6-element array into a 3×4 shape (12 elements needed).**
- NumPy repeats the array elements to fill the new size.

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6])
arr.resize((3, 4))
print(arr)
```

- **Output**[[1 2 3 4] [5 6 0 0] [0 0 0 0]]

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6])
arr.resize((3, 4))
print(arr,'\n')

print("numpy.resize")
print(np.resize(arr,(3,4)))
```

Output

```
[[1 2 3 4]
 [5 6 0 0]
 [0 0 0 0]]

numpy.resize
[[1 2 3 4]
 [5 6 0 0]
 [0 0 0 0]]
```

- **Example 3:**
- This example resizes an array into a 2×2 shape.
- Since fewer elements are required, the extra values are removed.

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50])
arr.resize((2, 2))
print(arr)
```

- **Output**[[10 20] [30 40]]

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50])
arr.resize((2, 2))
print(arr,'\n')

print("numpy.resize")
print(np.resize(arr,(3,4)))
```

Output

```
[[10 20]
 [30 40]]

numpy.resize
[[10 20 30 40]
 [10 20 30 40]
 [10 20 30 40]]
```

Random sampling in numpy | randint() function

- The **numpy.random.randint()** function is used to generate random integers within a specified range.

- It allows you to **create arrays of any shape filled with random integer values,** making it useful in simulations, testing, and numerical experiments.

- **Example:**
  - **Input:** Generate integers between 0 and 5

    Output: [3 1 4 0 2]

- **Explanation:** Each value is a **random integer from the interval [0, 5].**

- **Syntax**
- numpy.random.randint(low, high=None, size=None, dtype=int)

- **Parameters:**
  - **low: lowest integer** that can appear in the output.
  - **high(Optional): Upper limit (exclusive).**
    - If omitted, range becomes [0, low).
  - **size(Optional): Shape of the output array** (e.g., 5, (2,3), (2,3,4)).
  - **dtype(Optional):** Data type of the returned numbers.
    - Default is integer.

Examples

- **Example 1:**
- This example generates **five random integers between 0 and 4,** stored in a one-dimensional array.

```
import numpy as np
arr = np.random.randint(0, 5, size=5)
print(arr)
```

- **Output**[2 2 3 3 4]

- **Example 2:**
- This example **creates a 2×3 matrix of random integers ranging from 0 to 9**.

```
import numpy as np
arr = np.random.randint(0, 10, size=(2, 3))
print(arr)
```

Output[[7 9 8] [9 1 1]]

- **Explanation:**
  - size=(2, 3): **creates 2 rows and 3 columns.**
  - 0 to 10: upper limit 10 is excluded.

Contd..

- **Example 3:**
- This example produces a 3D array (2×2×4) with values between 5 and 15.

```
import numpy as np
arr = np.random.randint(5, 15, size=(2, 2, 4))
print(arr)
```

-

Output

```
[[[ 8 7 12 12]
 [ 6 13 13 6]]
 [[11 6 6 7]
 [ 7 9 6 8]]]
```

Pandas

- Pandas ((stands for Python Data Analysis)) is an **open-source Python library** used for data manipulation, analysis and cleaning.

- It provides fast and flexible tools to work with tabular data, similar to **spreadsheets or SQL tables.**

- Pandas is used in data science, machine learning, finance, analytics and automation because it **integrates smoothly with other libraries** such as:

- NumPy: numerical operations
- Matplotlib and Seaborn: data visualization
- SciPy: statistical analysis
- Scikit-learn: machine learning workflows

What is Pandas Used for?

- **Reading and writing data from various file for**mats like CSV, Excel and SQL databases.

- Cleaning and preparing data (handling missing values, filtering, removing duplicates).

- **Merging, joining, and reshaping datasets.**

- Performing statistical analysis and descriptive statistics.

- **Visualizing data quickly.**

Installation

- Before using Pandas, make sure it is installed:
  - ***pip install pandas***

- After the Pandas have been **installed in the system we need to import the library.**
- This module is imported using:
  - ***import pandas as pd***

- **Note: pd is just an alias for Pandas**. It's not required but using it makes the code shorter when calling methods or properties.

Data Structures in Pandas

- Pandas provides **two data structures for manipulating data which are as follows:**
- **1. Pandas Series**
- A Pandas Series is like **a column in a table.**

- A Pandas Series **is one-dimensional labeled array capable** of holding data of any type (integer, string, float, Python objects etc.).

- The **axis labels are** collectively called **indexes**.

- Series is **created by loading the datasets from existing storage** which can be a **SQL database, a CSV file or an Excel file.**

```
import pandas as pd

import numpy as np


s = pd.Series()

print("Pandas Series: ", s)

data = np.array(['g', 'e', 'e', 'k', 's'])


s = pd.Series(data)

print("Pandas Series:\n", s)
```

Output

```
Pandas Series:  Series([], dtype: object)
Pandas Series:
 0    g
1    e
2    e
3    k
4    s
dtype: object
```

- **2. Pandas DataFrame**
- Pandas DataFrame is a **two-dimensional data structure** with labeled axes (rows and columns).
- A DataFrame is a **two-dimensional, size-mutable and potentially heterogeneous tabular data structure** with labeled axes (rows and columns).

- Pandas DataFrame comes is a powerful tool that **allows us to store and manipulate data in a structured way**, similar to an Excel spreadsheet or a SQL table.

- A DataFrame is similar to a **table with rows and columns.**
- It helps in handling large amounts of **data, performing calculations, filtering information with ease.**
- It is created by **loading the datasets from existing storage** which can be a **SQL database, a CSV file or an Excel file.**
- It can be created from lists, dictionaries, a list of dictionaries etc.

Example 1 –

```
import pandas as pd

df = pd.DataFrame()
print(df)
lst = ['Geeks', 'For', 'Geeks', 'is', 'portal', 'for', 'Geeks']
  #load list into a DataFrame object:
df = pd.DataFrame(lst)
print(df)
```

Output

```
Empty DataFrame
Columns: []
Index: []
        0
0   Geeks
1     For
2   Geeks
3      is
4  portal
5     for
6   Geeks
```

Example 2 –

```
import pandas as pd

    data = {
      "calories": [420, 380, 390],
      "duration": [50, 40, 45]
    }

    #load data into a DataFrame object:
    df = pd.DataFrame(data)

    print(df)
```

```
   calories  duration
0       420        50
1       380        40
2       390        45
```

Operations in Pandas

- Pandas provides essential **operations for working with structured data efficiently.**

- The sections below introduce the most **commonly used functionalities with short explanations and simple examples.**

Pandas Basic Operations

Pandas simplifies data handling by enabling efficient preprocessing, cleaning, transformation, and visualization.

DATA PREPROCESSING : Handle missing values

DATA CLEANING : Remove duplicates and fix inconsistencies

DATA TRANSFORMATION : Filter, sort & modify data

DATA VISUALIZATION : Represent data graphically

1. Loading Data: This **operation** **reads data** **from files** such as CSV, Excel or JSON into a DataFrame.

2. *You can download the data.csv file from here*

import pandas as pd

df = pd.read_csv("data.csv")

print(df.head())

df.head() shows the first 5 rows of the data.

The head() method returns the **headers and a specified number of rows, starting from the top.**

	name	age	city	category	sales	a	b
0	John	25	Delhi	A	200	5	10
1	Riya	22	Mumbai	B	120	2	3
2	Aman	30	Pune	A	250	7	8
3	Neha	27	Kolkata	C	300	4	6
4	Arjun	35	Jaipur	B	200	1	9

Reading an excel file using Python

https://www.geeksforgeeks.org/python/reading-excel-file-using-python/

- **Python** provides several modules to interact with Excel files.

- These **libraries** make it easy to read, write, and modify Excel data depending on your needs.

- The most commonly used libraries are:
  - **pandas:** fast, high-level data analysis
  - **openpyxl:** fine-grained control of .xlsx files
  - **xlwings:** connects directly with Excel for automation

Using pandas

- **Pandas** is the most **popular library for data analysis in Python.**

- It can **quickly load Excel files into a DataFrame,** making it easy to explore and manipulate tabular data.

Using pandas

- Below is an image of the sample Excel data we are going to use in this article, [click here to download it](#).

	A	B	C	D	E	F
1		Name	Age	Stream	Percentage	
2	0	Ankit	18	Math	95	
3	1	Rahul	19	Science	90	
4	2	Shaurya	20	Commerce	85	
5	3	Aishwarya	18	Math	80	
6	4	Priyanka	19	Science	75	

- import pandas as pd

  df = pd.read_excel('student_data.xlsx')

  print(df)

- **Explanation:**
- **import pandas as pd:** imports the pandas library.
- **pd.read_excel("student_data.xlsx"):** reads Excel file named student_data.xlsx into a DataFrame.
- **df.head():** shows first 5 rows of the data.

		Name	Age	Stream	Percentage
0	0	Ankit	18	Math	95
1	1	Rahul	19	Science	90
2	2	Shaurya	20	Commerce	85
3	3	Aishwarya	18	Math	80
4	4	Priyanka	19	Science	75

2. Viewing and Exploring Data:

After loading data, it is important to understand its structure and content.

This **methods** allow you to inspect rows, summary statistics and metadata.

print(df.info())

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 7 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   name       5 non-null      object
 1   age        5 non-null      int64
 2   city       5 non-null      object
 3   category   5 non-null      object
 4   sales      5 non-null      int64
 5   a          5 non-null      int64
 6   b          5 non-null      int64
dtypes: int64(4), object(3)
memory usage: 412.0+ bytes
None
```

https://www.w3schools.com/python/pandas/ref_df_info.asp

- The **info() method prints** information **about the DataFrame**.
- The **information contains** the number of columns, column labels, column data types, memory usage, range index, and the number of cells in each column (non-null values).

- **Syntax**
- ***dataframe*.info(verbose,buf,max_cols,memory_usage,show_counts,null_counts)**

- **Return Value**
- None.
- **The info() method** does not return any value, it prints the information.

- **Syntax**
- ***dataframe*.info(verbose,buf,max_cols,memory_usage,show_counts,null_counts)**

Parameter	**Value**	**Description**
verbose	True|False	Optional. Whether to print all the information or not
buf	buffer	Optional. Outputs to buffer instead of to the sys.stdout. Useful when writing to files
max_cols	int	Optional. Specifies the number of columns you want information from
memory_usage	True|False|*string*	Optional. Specifies whether to print the memory usage or not, or 'deep' to do a real calclulation of the memory usage (at a high computer resource cost) instead of an estimate based on dtypes and number of rows (lower cost).
show_counts	True|False	Optional. Specifies whether to print the non-null counts or not

Pandas DataFrame describe() Method

- **The describe() method** returns description of the data in the DataFrame.

- If **the DataFrame** contains numerical data, the description contains these information for each column:
- **count** - The number of not-empty values.
  mean - The average (mean) value.
  std - The standard deviation.
  min - the minimum value.
  25% - The 25% percentile*.
  50% - The 50% percentile*.
  75% - The 75% percentile*.
  max - the maximum value.

- ***Percentile meaning**: how many of the values are less than the given percentile.

- **Syntax**:- ***dataframe*.describe(percentiles, include, exclude, datetime_is_numeric)**

- **Parameters**:- The percentile, include, **exclude, datetime_is_numeric parameters are keyword arguments.**

- **Return Value**
- A DataFrame object with **statistics for each row.**

Parameter	**Value**	**Description**
percentile	*numbers between: 0 and 1*	Optional, a list of percentiles to include in the result, default is : [.25, .50, .75].
include	None 'all' *datatypes*	Optional, a list of the data types to allow in the result
exclude	None 'all' *datatypes*	Optional, a list of the data types to disallow in the result
datetime_is_numeric	True False	Optional, default False. Set to True to treat datetime data as numeric

- **Example**
- Return a **statistically description of the data in the DataFrame:**
- import pandas as pd

```
data = [[10, 18, 11], [13, 15, 8], [9, 20, 3]]

df = pd.DataFrame(data)

print(df.describe())
```

```
              0          1          2
count  3.000000   3.000000   3.000000
mean  10.666667  17.666667   7.333333
std    2.081666   2.516611   4.041452
min    9.000000  15.000000   3.000000
25%    9.500000  16.500000   5.500000
50%   10.000000  18.000000   8.000000
75%   11.500000  19.000000   9.500000
max   13.000000  20.000000  11.000000
```

Pandas DataFrame head() Method

- The **head() method** returns a specified number of rows, string from the top.
- The **head() method** returns the **first 5 rows if a number is not specified.**

- **Syntax** :- *dataframe*.head(*n*)
- **Parameters:--**

Parameter	Description
n	Optional. The number of rows to return. Default value is 5.

- **Return Value:-** A DataFrame with **headers and the specified number of rows.**

- **Example**
- Return the **first 5 rows of the DataFrame**.
- And also 10 rows of column
- In this example we use a .csv file called data.csv

```
import pandas as pd
df = pd.read_csv('data.csv’)
# first 5 rows of the DataFrame
print(df.head())

series = df["Pulse"]
# first 10 rows of column - Pulse

top = series.head(n = 10)
print(" Dataset (First 10 Rows):")
print(top)
```

Pandas DataFrame tail() Method

- **The tail() method** returns a **specified number of last rows.**

- **The tail() method** returns the **last 5 rows if a number is not specified**.

- **Syntax**:- *dataframe*.tail(*n*)

Parameter	**Description**
n	Optional. The number of rows to return. Default value is 5.

- **Return Value :-** A DataFrame with headers and the specified number of rows.

- **Example**
- Return the last 5 rows of the DataFrame. In this example we use a .csv file called data.csv
- import pandas as pd

  df = pd.read_csv('data.csv')

  print(df.tail())

```
     Duration  Pulse  Maxpulse  Calories
164        60    105       140     290.8
165        60    110       145     300.4
166        60    115       145     310.2
167        75    120       150     320.4
168        75    125       150     330.4
```

- **Example**

- import pandas as pd
- **df = pd.read_csv('data.csv')**
- print(df.tail(10))

```
     Duration  Pulse  Maxpulse  Calories
159        30     80       120     240.9
160        30     85       120     250.4
161        45     90       130     260.4
162        45     95       130     270.0
163        45    100       140     280.9
164        60    105       140     290.8
165        60    110       145     300.4
166        60    115       145     310.2
167        75    120       150     320.4
168        75    125       150     330.4
```

value_counts() in Pandas

https://www.geeksforgeeks.org/python/how-to-extract-the-value-names-and-counts-from-value_counts-in-pandas/

- **The panda library** is equipped with a **number of useful functions** for 'value_counts' is one of them.

- This function returns the **counts of unique items in a pandas data frame.**

- **Syntax:** ***<object>.value_count()***
- **Approach:**
  - Import Required module.
  - Make the DataFrame
  - Process using value_count()
  - Display data

- **Example 1:** To print all the **unique country and the first country name** in the list.
  - ***tolist()*** *function return a list of the values.*
- ***Syntax:*** *Index.tolist()*

  Parameters : *None*

  Returns : *list*

https://www.geeksforgeeks.org/python/how-to-extract-the-value-names-and-counts-from-value_counts-in-pandas/

```
import pandas as pd
import matplotlib.pyplot as plt

# Make example dataframe
df = pd.DataFrame([(1, 'Germany'),
                   (2, 'France'),
                   (3, 'Indonesia'),
                   (4, 'France'),
                   (5, 'France'),
                   (6, 'Germany'),
                   (7, 'UK'),
                   (8, 'India'),
                   (9, 'India'),
                   (10, 'Germany')
                   ],
                  columns=['groupid', 'country'],
                  index=['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j'])


# print all unique country name in the list
su1 = df['country'].value_counts().index.tolist()
print(su1)

# print 1st unique country name present in a list
su2 = df['country'].value_counts().index.tolist()[0]
print(su2)
```

- **Example 2:** To **print all the unique values** of the column and the first value of the column.
  - ***value_count()*** *counts Unique Occurrences of Values in a Column*
- **Syntax**: Index.value_count()

  Parameters: None

  Returns: the **count of occurrences of each of the unique values** in this column.

```
import pandas as pd
import matplotlib.pyplot as plt

# Make example dataframe
df = pd.DataFrame([(1, 'Germany'),
                   (2, 'France'),
                   (3, 'Indonesia'),
                   (4, 'France'),
                   (5, 'France'),
                   (6, 'Germany'),
                   (7, 'UK'),
                   (8, 'India'),
                   (9, 'India'),
                   (10, 'Germany')
                   ],
                  columns=['groupid', 'country'],
                  index=['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j'])


# print country name and counts
su3 = df['country'].value_counts()
print(su3)

# print 1st country count in a list
su4 = df['country'].value_counts()[0]
print(su4)
```

Matplotlib

https://www.geeksforgeeks.org/python/matplotlib-tutorial/

- Matplotlib is an open-source **visualization library** for the Python programming language, widely used for creating static, animated and interactive plots.

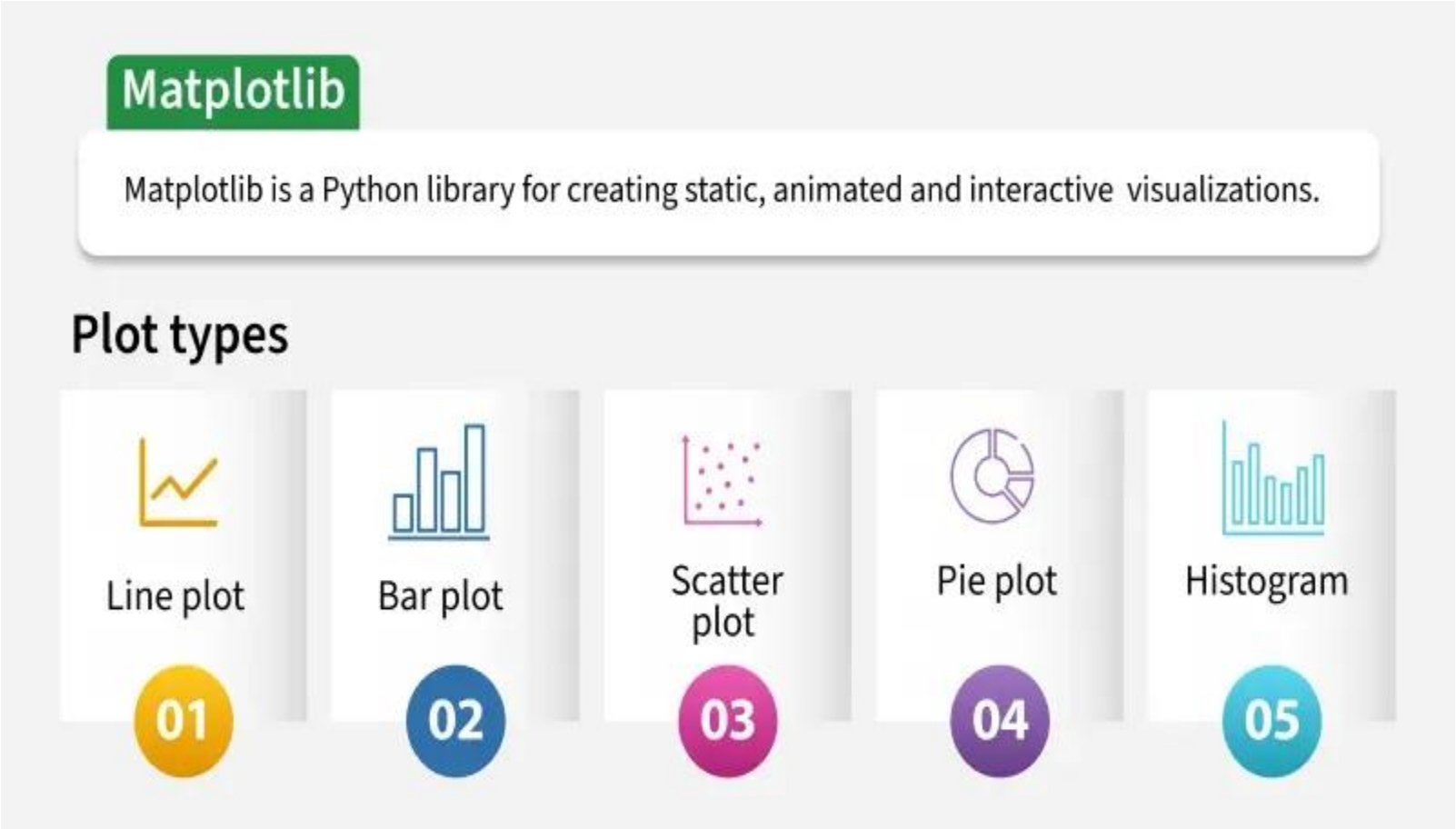
- It provides an **object-oriented API** for embedding plots into applications using **general-purpose GUI toolkits** like Tkinter, Qt, GTK and wxPython.

- It offers a **variety of plotting functionalities**, including line plots, bar charts, histograms, scatter plots and 3D visualizations.

- Created by John D. Hunter in 2003, Matplotlib has become a **fundamental tool for data visualization in Python**, extensively used by data scientists, researchers and engineers worldwide.

- It provides **flexible and customizable plotting functions** that help in understanding data patterns, trends and relationships effectively.

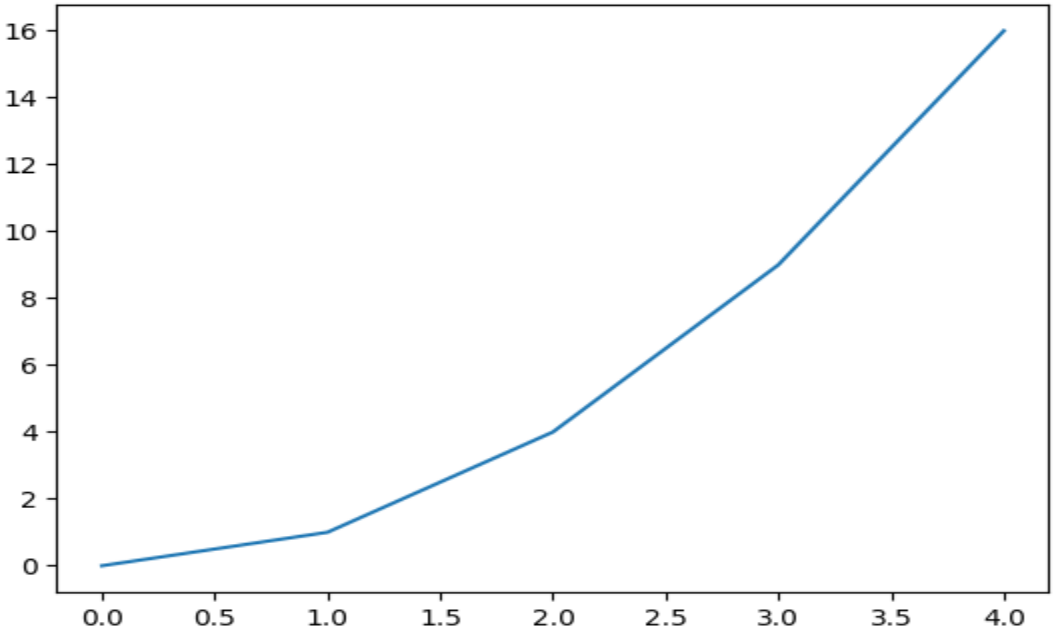
Contd..


Matplotlib
Matplotlib is a Python library for creating static, animated and interactive visualizations.
Plot types
Line plot
01
Bar plot
02
Scatter plot
03
Pie plot
04
Histogram
05

Contd..

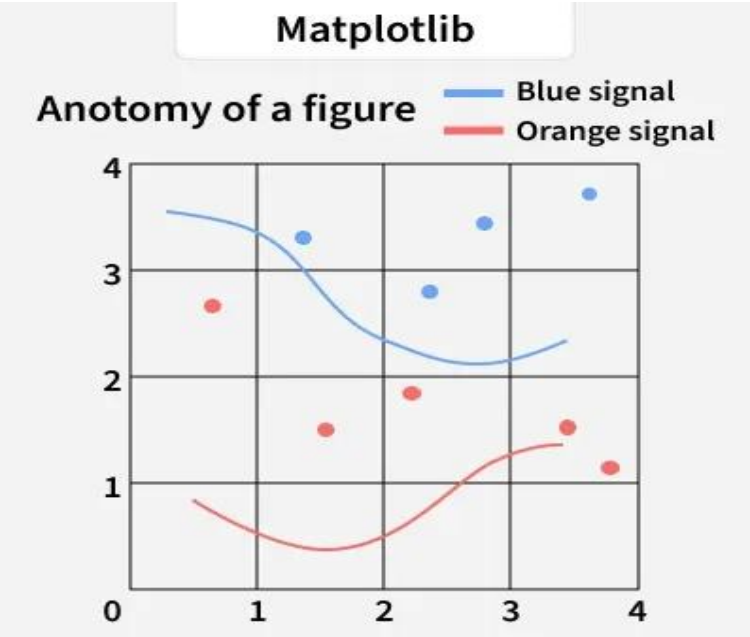
- **Example of a Plot in Matplotlib:**
- Let's create a **simple line plot using Matplotlib**, showcasing the ease with which you can visualize data.

```
import matplotlib.pyplot as plt

x = [0, 1, 2, 3, 4]
y = [0, 1, 4, 9, 16]

plt.plot(x, y)
plt.show()
```



Components or Parts of Matplotlib Figure

- **Anatomy of a Matplotlib Plot:**
- The **key components of a Matplotlib plot**, including
  - figures,
  - axes,
  - titles, and
  - legends, essential for effective data visualization.



- A legend is an **area describing the elements of the graph**.

- In the Matplotlib library, there's a function called **legend()** which is used **to place a legend on the axes.**

- In the Figure given --the legend provides color-based labels **"blue signal" and "orange signal** for clarity.

- The parts of a Matplotlib figure include (as shown in the figure above):
- **Figure**:
  - The **overarching container** that holds all plot elements, acting as the **canvas for visualizations**.
  - (In Python, an **overarching container** refers to a **data structure (object) that holds, organizes, and manages other objects (elements or data types) within i**t.
  - It acts as a **single, higher-level unit—a "container" or "box"—used to store multiple data items together,** which can then be manipulated, iterated over, or accessed collectively.)
- **Axes**:
  - The **areas within the figure** where data is plotted;
  - **each figure** can contain multiple axes.

- **Axis**: Represents the
  - **x-axis and y-axis,**
  - defining limits,
  - tick locations, and
  - **labels for data interpretation.**

- **Lines and Markers**:
  - Lines **connect data points** to show trends,
  - while **markers denote individual data points** in plots like scatter plots.

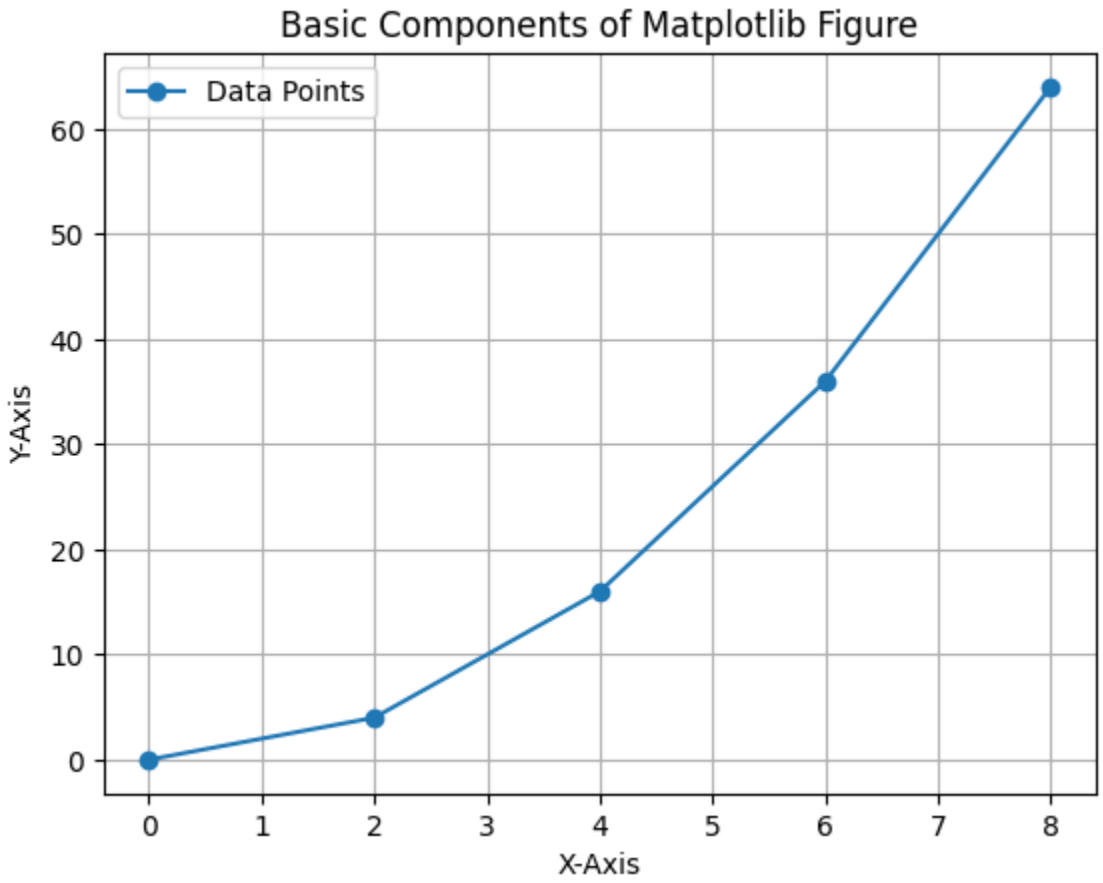
- **Title and Labels**:
  - The **title** provides context for the plot,
  - while **axis labels** describe what data is being represented on each axis.

Matplotlib Pyplot

- Pyplot is a **module** within Matplotlib that provides a **MATLAB-like interface for making plots.**
- It **simplifies the process of adding plot elements** such as lines, images, and text to the axes of the current figure.
- **Steps to Use Pyplot:**
  - **Import Matplotlib:** Start by importing matplotlib.pyplot as plt.
  - **Create Data:** Prepare **your data in the form of lists or arrays.**
  - **Plot Data**: Use plt.plot() to create the plot.
  - **Customize Plot**: **Add titles, labels, and other elements** using methods like plt.title(), plt.xlabel(), and plt.ylabel().
  - **Display Plot**: **Use plt.show() to display the plot.**

- Let's **visualize a basic plot, and understand basic components** of matplotlib figure:

```
import matplotlib.pyplot as plt

x = [0, 2, 4, 6, 8]
y = [0, 4, 16, 36, 64]

fig, ax = plt.subplots()
ax.plot(x, y, marker='o', label="Data Points")

ax.set_title("Basic Components of Matplotlib Figure")
ax.set_xlabel("X-Axis")
ax.set_ylabel("Y-Axis")


plt.show()
```


Basic Components of Matplotlib Figure
Data Points
Y-Axis
X-Axis
0
10
20
30
40
50
60
0
1
2
3
4
5
6
7
8

Key Features of Matplotlib

- **Versatile Plotting**: Create a **wide variety of visualizations**, including line plots, scatter plots, bar charts, and histograms.

- **Extensive Customization**: **Control every aspect of your plots**, from colors and markers to labels and annotations.

- **Seamless Integration with NumPy**: **Effortlessly plot data arrays** directly, enhancing data manipulation capabilities.

- **High-Quality Graphics**: **Generate publication-ready plots** with precise control over aesthetics.

- **Cross-Platform Compatibility**: Use **Matplotlib on Windows, macOS, and Linux** without issues.

- **Interactive Visualizations**: Engage with your data dynamically through interactive plotting features.

What is Matplotlib Used For?

- **Matplotlib is a Python library** for data visualization, primarily used to create **static, animated, and interactive plots.**

- It provides a wide range of plotting functions to visualize data effectively.

- **Key Uses of Matplotlib:**
  - **Basic Plots: Line plots, bar charts, histograms, scatter plots**, etc.
  - **Statistical Visualization:** Box plots, error bars, and density plots.
  - **Customization:** Control over **colors, labels, gridlines, and styles**.
  - **Subplots & Layouts:** Create multiple plots in a single figure.
  - **3D Plotting: Surface plots and 3D scatter plots** using mpl_toolkits.mplot3d.
  - **Animations & Interactive Plots:** Dynamic visualizations with FuncAnimation.
  - **Integration:** Works well with **Pandas, NumPy and Jupyter Notebooks.**

https://www.geeksforgeeks.org/data-visualization/matplotlib-step-by-step-guide/

- **Visualizing** data helps us understand and share information more effectively.

- In Python **Matplotlib** is one of the best tools for creating visualizations.

- It's **powerful, flexible** and lets you make many types of plots, from simple line charts to advanced animations.

- **Setting Up Matplotlib**
- Before using Matplotlib, ensure you have it installed.
- We can install it using pip:
  - ***pip install matplotlib***

- Once installed, we can import it into your Python script:
  - ***import matplotlib.pyplot as plt***

Essential Role of Pyplot in Matplotlib

- **Pyplot** is a submodule of the Matplotlib library in Python **providing a beginner-friendly tool for creating visualizations with minimal code**.

- It helps **transform dull data into engaging and interactive plots**, making it easier to analyze and draw meaningful insights for informed decision-making.

- ***Syntax:***
  - ***matplotlib.pyplot.plot()***

Creating Different Types of Plots

- **In data visualization**, creating various **types of plots is essential for effectively** conveying insights from data.

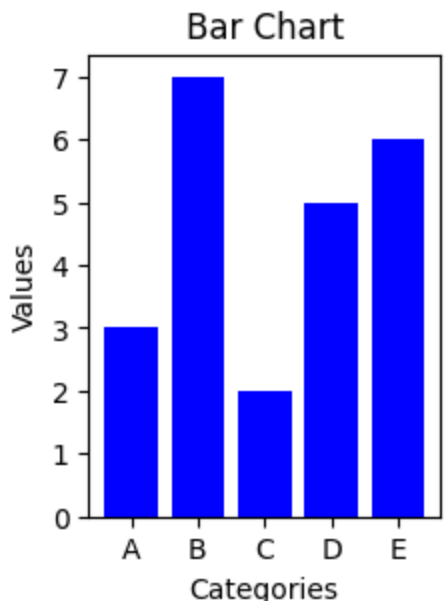
- Below, we'll explore how to create different types of plots using Matplotlib.

- Each **type of plot in Matplotlib is designed** to address specific analytical scenarios, making them **value able tools for businesses looking** for getting insights for better decision making in the industry.

Different Types of Plots in Matplotlib

- **Matplotlib offers** a wide range of plot types to suit various data visualization needs.

- Here are some of the most commonly used types of plots in Matplotlib:
  - Line Graph
  - Bar Chart
  - Histogram
  - Scatter Plot
  - Pie Chart
  - 3D Plot



1. Line Graph

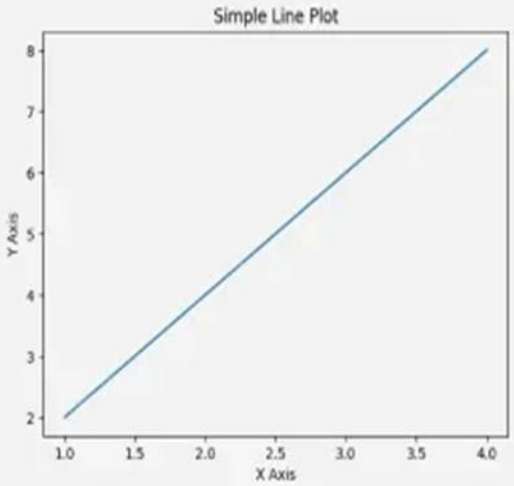
- Line graphs are commonly **used to visualize trends** over time or relationships between variables.

Line plot

A line plot connects data points with straight lines to show trends. In Matplotlib, the plot() function is used to create it.

Common uses of line plots:

- Tracking time series data (e.g., stock prices, temperature).
- Comparing multiple datasets.
- Identifying patterns and trends in continuous data.



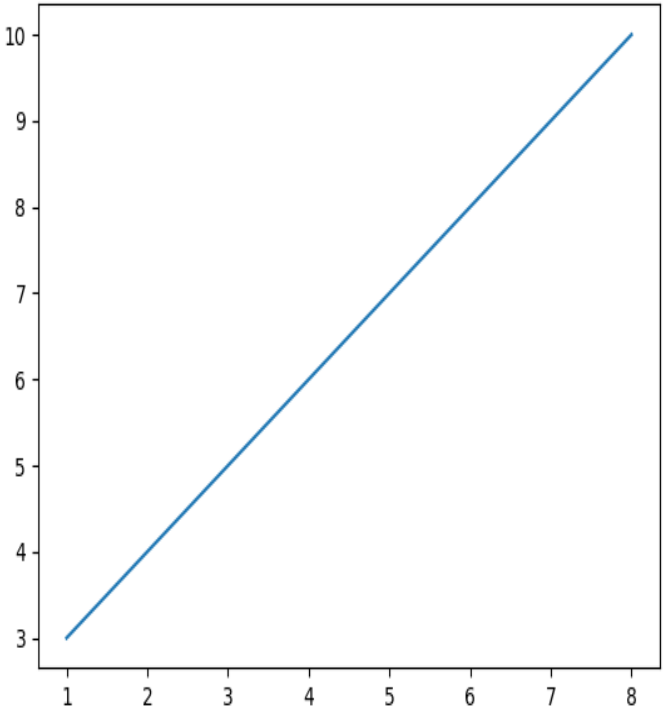
Matplotlib Plotting

- **Plotting x and y points**
- The **plot() function** is used to **draw points (markers) in a diagram.**
- **By default**, the plot() function draws a line from point to point.
- The **function takes parameters** for specifying points in the diagram.
- **Parameter 1** is an **array** containing the points on the **x-axis**.
- **Parameter 2** is an **array** containing the points on the **y-axis**.
- If we need to plot a line from (1, 3) to (8, 10), we have to pass two arrays [1, 8] and [3, 10] to the plot function.

- **Example**
- Draw a line in a diagram from position (1, 3) to position (8, 10):
- import matplotlib.pyplot as plt
  import numpy as np

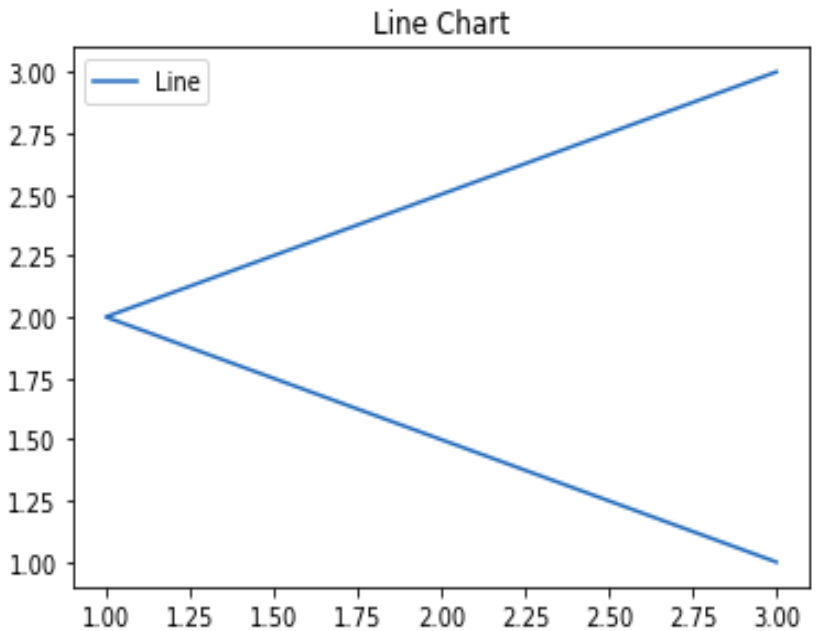
  xpoints = np.array([1, 8])
  ypoints = np.array([3, 10])

  plt.plot(xpoints, ypoints)
  plt.show()



- **Example**

```
import matplotlib.pyplot as plt
x = [3, 1, 3]
y = [3, 2, 1]

plt.plot(x, y)
plt.title("Line Chart")

plt.legend(["Line"])
plt.show()
```



Simply using **plt.plot helped us to create an chart.**

2. Bar chart

- A **bar plot** or **bar chart is a graph** that represents the category of data with rectangular bars with **lengths and heights** that is proportional to the values which they represent.

- The bar plots can **be plotted horizontally or vertically.**

- A bar chart describes the comparisons between the discrete categories.

- It **can be created** using the **bar()** method.

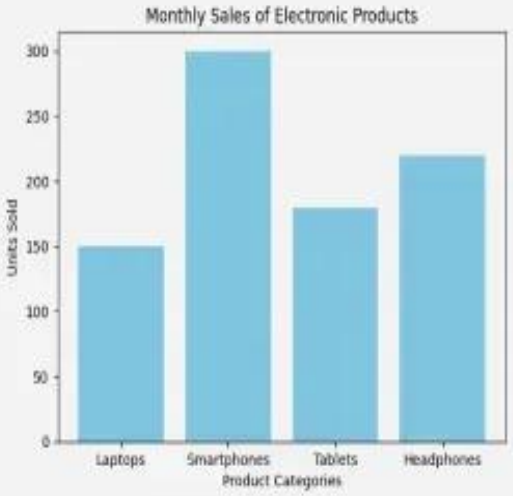
2. Bar chart

Bar Plot

A bar plot uses rectangular bars to represent categorical data, with height indicating value. In Matplotlib, it's created using the bar() function.

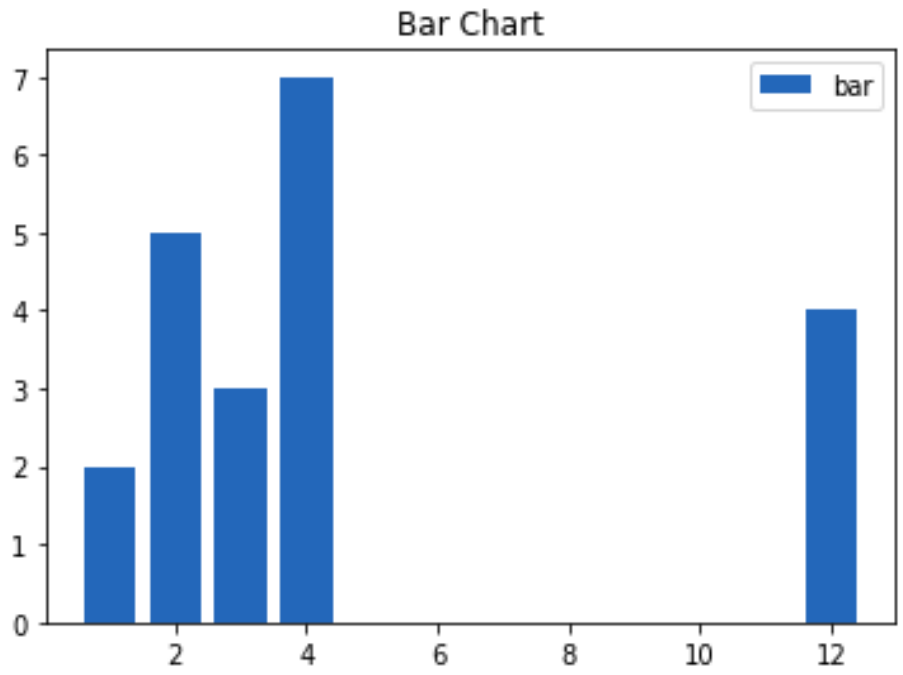
Common uses of bar plots:

- Comparing categories or groups.
- Showing distributions and frequencies.
- Visualizing differences in data values.



Example:-

```
import matplotlib.pyplot as plt
x = [3, 1, 3, 12, 2, 4, 4]
y = [3, 2, 1, 4, 5, 6, 7]

plt.bar(x, y)

plt.title("Bar Chart")

plt.legend(["bar"])
plt.show()
```



3. Plotting Histogram

- **Histogram** is basically used to represent data in the form of some groups.

- It is a **type of bar plot** where the **X-axis represents the bin ranges** while the Y-axis gives information about frequency.

- The hist() function **will use an array of numbers to create a histogram**, the array is sent into the function as an argument.

- To create a histogram the
  - first step is to **create a bin of the ranges,**
  - then distribute the whole range of the values into a series of intervals and
  - **count the values which fall into each of the intervals.**

- **Bins are clearly identified** as consecutive, non-overlapping intervals of variables.

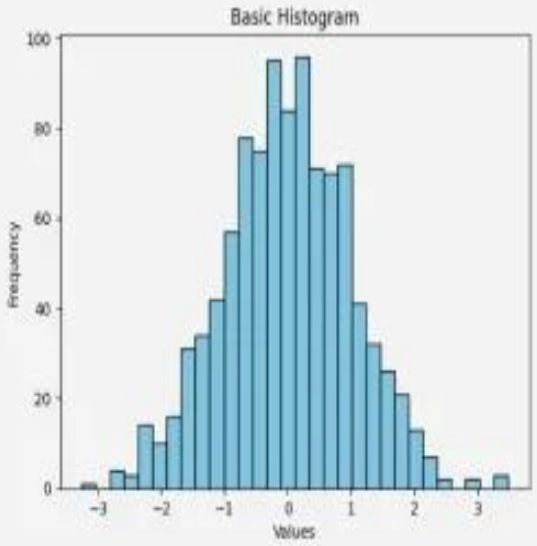
- (In Python, when creating a histogram using libraries like Matplotlib or NumPy, **"bins" are the discrete intervals or ranges used to group continuous numerical data**. Each bin collects data points that fall within its specific range, and the height of the corresponding bar in the histogram represents the frequency (count) of those data points.)

Histogram

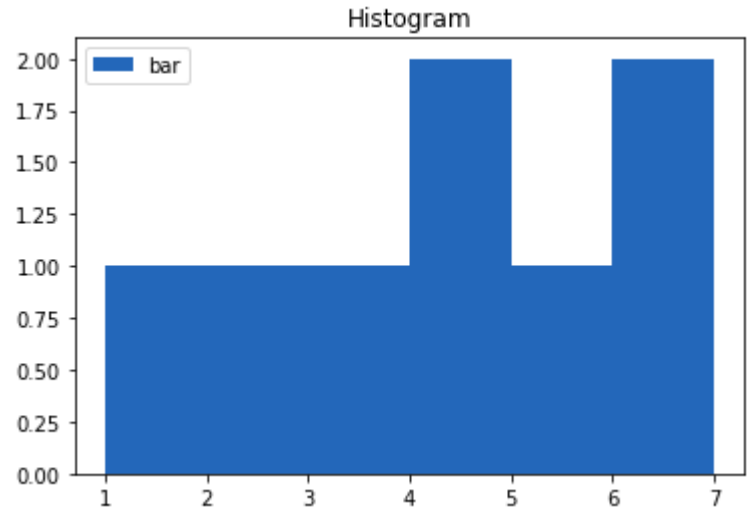
A histogram represents the distribution of numerical data using bars, where height indicates frequency. In Matplotlib, it's created using the hist() function.

Common uses of histograms:

- Visualizing the distribution of continuous data.
- Identifying patterns, skewness, and outliers.
- Understanding frequency distribution in datasets.



```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5, 6, 7, 4]

plt.hist(x, bins = [1, 2, 3, 4, 5, 6, 7])

plt.title("Histogram")
plt.legend(["bar"])
plt.show()
```



The data points in x are grouped into these intervals, and the height of each bar in the histogram represents the number of data points (frequency) that fall into its corresponding bin.

x = [1, 2, 3, 4, 5, 6, 7, 4] (The data)

bins = [1, 2, 3, 4, 5, 6, 7] (The boundaries)

- The resulting bins are:

  [1, 2): Contains 1 data point (1)

  [2, 3): Contains 1 data point (2)

  [3, 4): Contains 1 data point (3)

  [4, 5): Contains 2 data points (4, 4)

  [5, 6): Contains 1 data point (5)

  [6, 7): Contains 1 data point (6)
- [7] : The value 7 is not included in any bin because the list of bins ends at 7. The plt.hist() function, by default, creates bins where the lower bound is inclusive and the upper bound is exclusive.
- Therefore, the last data point 7 will not be displayed in the histogram with this specific bins list.

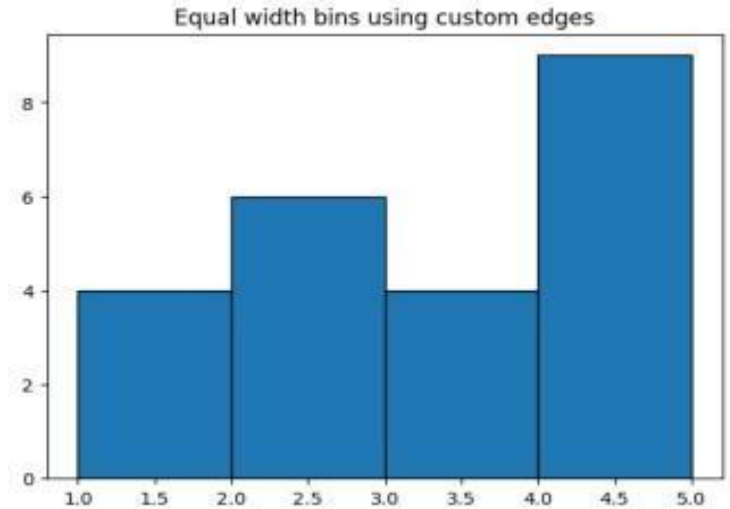
- **Using list of edges**
- You manually define each bin edge using a list.

- This gives full control over bin ranges, including unequal widths.

- Useful when data needs custom grouping or uneven intervals.

- **Example 1:** Equal width bins using custom edges

```
import matplotlib.pyplot as plt

a = [1, 2, 3, 2, 1, 2, 3, 2,
     1, 4, 5, 4, 3, 2, 5, 4,
     5, 4, 5, 3, 2, 1, 5]
```




```
plt.hist(a, bins=[1, 2, 3, 4, 5], edgecolor="black")
plt.title("Equal width bins using custom edges")
plt.show()
```

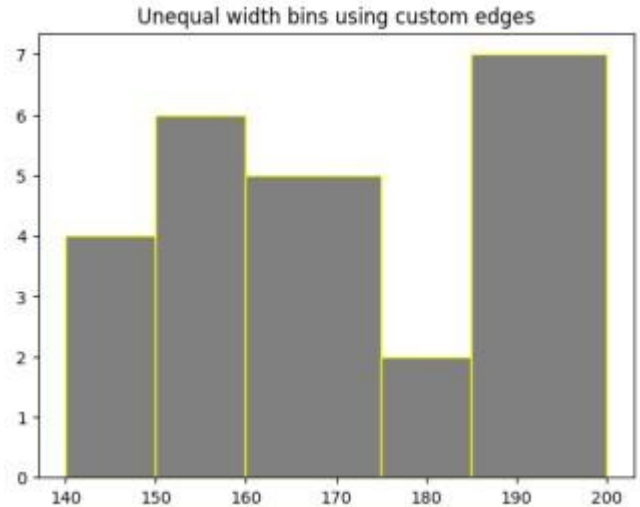
Explanation:

Setting **bins=[1, 2, 3, 4, 5]** defines custom bin edges, creating equal-width bins of size 1.

Matplotlib groups values based on these edges and counts their frequency for bar heights. edgecolor="black" outlines the bars for clear distinction.

- **Example 2:** Unequal width bins using custom edges

```
import matplotlib.pyplot as plt

a = [189, 185, 195, 149, 189, 147,
     154, 174, 169, 195, 159, 192,
     155, 191, 153, 157, 140, 144,
     172, 157, 181, 182, 166, 167]

plt.hist(a, bins=[140, 150, 160, 175, 185, 200],
         edgecolor="yellow", color="grey")
plt.title("Unequal width bins using custom edges")
plt.show()
```



Explanation:

bins list [140, 150, 160, 175, 185, 200] sets custom bin edges with unequal widths.

Matplotlib counts how many values fall into each interval. **edgecolor="yellow"** and **color="grey"** style the histogram for visual appeal.

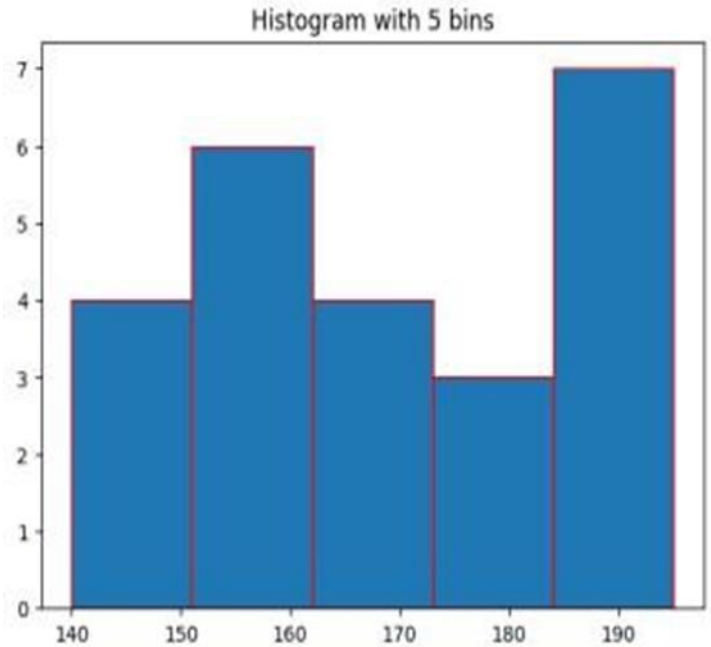
https://www.geeksforgeeks.org/python/bin-size-in-matplotlib-histogram/

- **Using integer bins**

- When you **pass an integer to the bins parameter** in Matplotlib, it automatically divides the entire range of data into that many equal-width bins.

- This approach **allows for quick and simple visualizations** without needing to manually specify bin edges or widths.

```
import matplotlib.pyplot as plt

a = [189, 185, 195, 149, 189, 147, 154,
    174, 169, 195, 159, 192, 155, 191,
    153, 157, 140, 144, 172, 157, 181,
    182, 166, 167]

plt.hist(a, bins=5, edgecolor="red")
plt.title("Histogram with 5 bins")
plt.show()
```



- **Explanation:**
- Setting **bins=5** divides the data range into 5 equal-width intervals.
- Matplotlib calculates the bin width and counts how many values fall into each bin to set bar heights. **edgecolor="red"** highlights bar borders for clarity.

4. Scatter Plot

- Scatter plots are **ideal for visualizing** the relationship between two continuous variables.

- Scatter plots help **identify patterns, correlations or clusters within data points.**

- The scatter() function plots one dot for each observation.

- It needs **two arrays of the same length**, one for the values of **the x-axis**, and one for values on **the y-axis:**

4. Scatter Plot

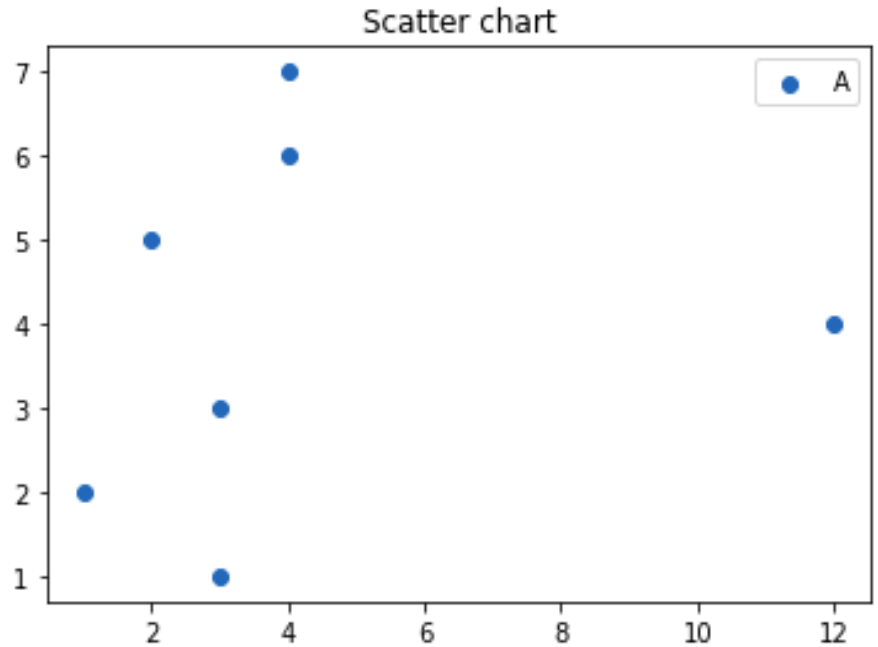
Scatter Plot

A scatter plot visualizes relationships between two variables using individual data points. In Matplotlib, it's created with the scatter() function.

Common uses of scatter plots:

- Showing relationships between two numerical variables.
- Identifying trends, clusters, and outliers.
- Visualizing correlations in data.

```
import matplotlib.pyplot as plt

x = [3, 1, 3, 12, 2, 4, 4]
y = [3, 2, 1, 4, 5, 6, 7]
plt.scatter(x, y)

plt.legend("A")

plt.title("Scatter chart")
plt.show()
```



Compare Plots

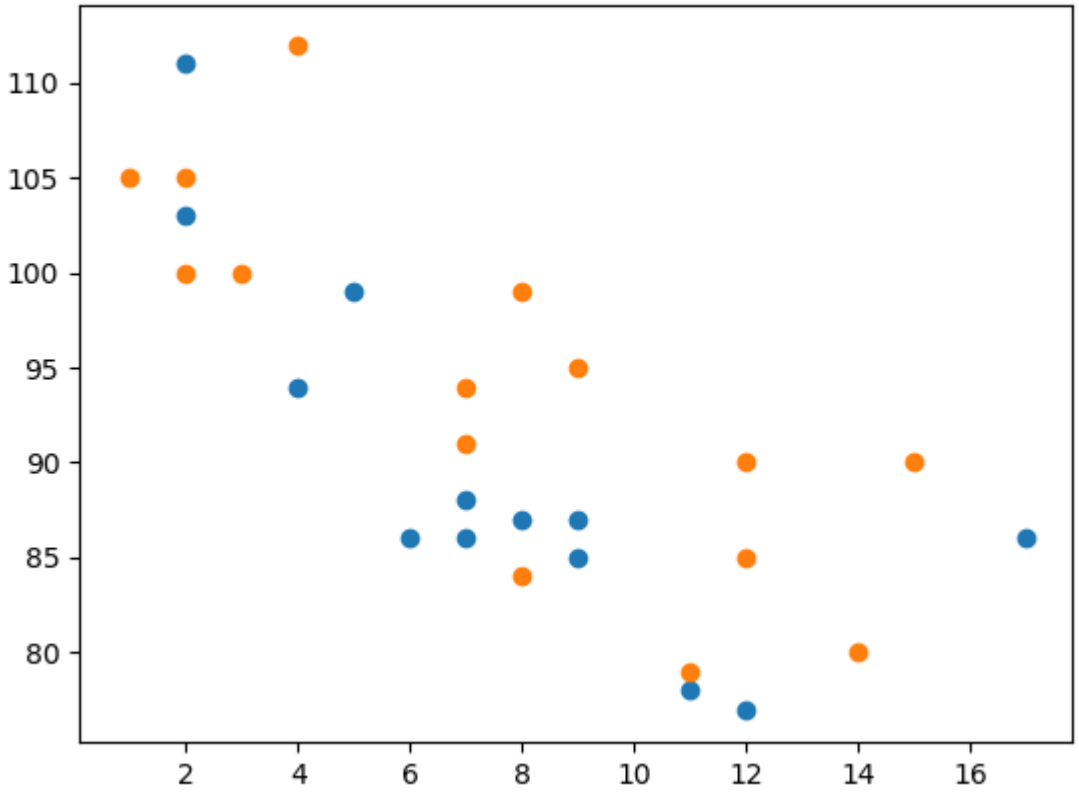
- **Example**
- **Draw two plots on the same figure:**
- import matplotlib.pyplot as plt
  import numpy as np

  #day one, the age and speed of 13 cars:
  x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
  y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
  plt.scatter(x, y)

  #day two, the age and speed of 15 cars:
  x = np.array([2,2,8,1,15,8,12,9,7,3,11,4,7,14,12])
  y = np.array([100,105,84,105,90,99,90,95,94,100,79,112,91,80,85])
  plt.scatter(x, y)

  plt.show()

- Result



5. Pie Chart

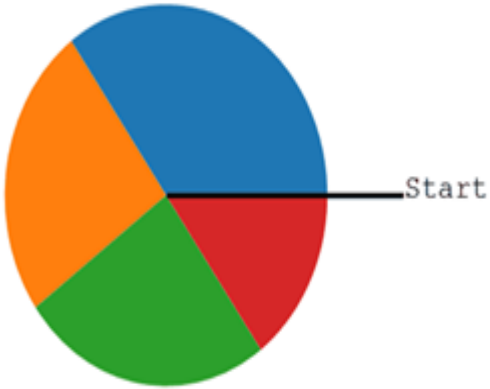
- **Pie Chart** is a **circular statistical plot** that can display only one series of data.

- The **area of the chart** is the total percentage of the given data.

- The **area of slices** of the pie represents the percentage of the parts of the data.

- The **slices of pie** are called wedges.
- By default the plotting of the first wedge starts from the x-axis and moves *counterclockwise*:

- The **area of the wedge** is determined by the length of the arc of the wedge.

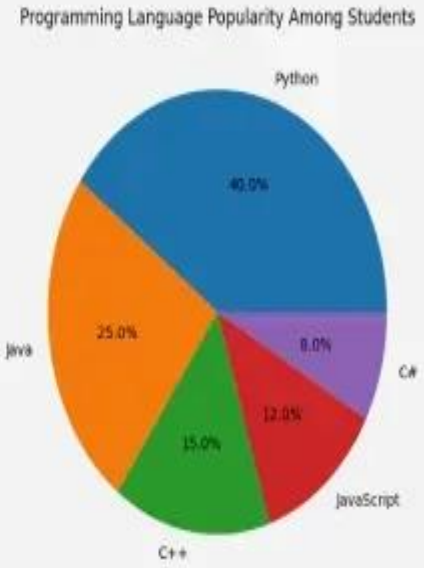


Pie plot

A pie plot represents data as slices of a circle, showing proportions of a whole. In Matplotlib, it's created using the pie() function.

Common uses of pie plots:

- Showing proportions or percentage distribution.
- Visualizing categorical data as parts of a whole.
- Comparing relative sizes of different categories.



- **Labels**
- Add **labels to the pie chart** with the labels parameter.
- The **labels parameter** must be an array with one label for each wedge:

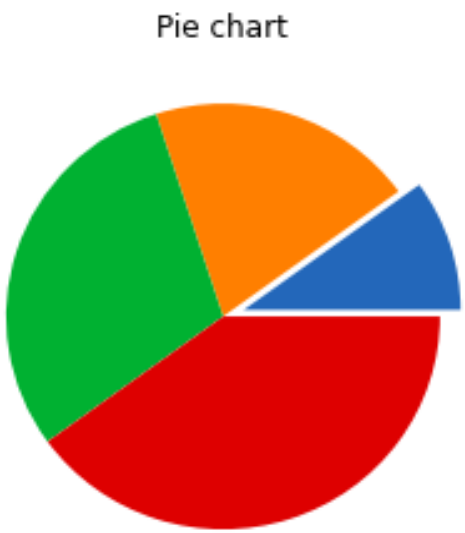
- **Explode**
- Maybe you want one of the wedges to stand out?
- The **explode parameter** allows you to do that.
- The **explode parameter, if specified**, and not None, must be an array with one value for each wedge.
- **Each value** represents how far from the center each wedge is displayed:

- **Colors**
- You can **set the color of each wedge** with the colors parameter.
- The **colors parameter, if specified**, must be an array with one value for each wedge:

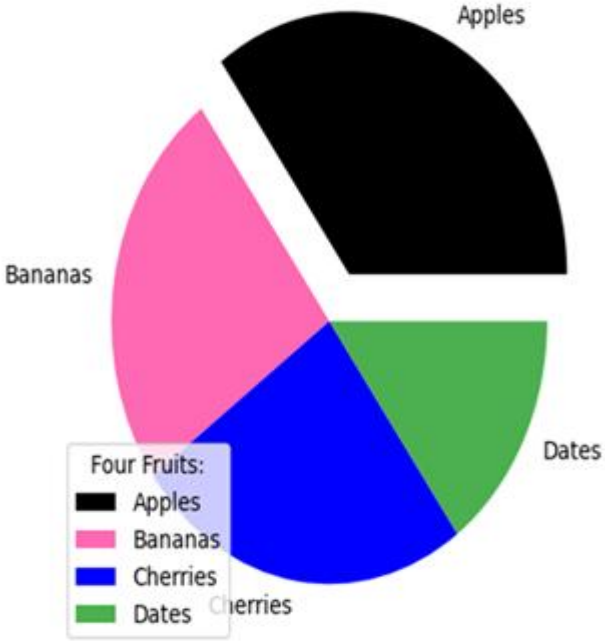
- **Legend**
- To add a list of explanation for each wedge, use the legend() function:

- **Legend With Header**
- To add a header to the legend, add the title parameter to the legend function.

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]


e  =(0.1, 0, 0, 0)


plt.pie(x, explode = e)


plt.title("Pie chart")

plt.show()
```



```
#Three lines to make our compiler able to draw:
import sys
import matplotlib
matplotlib.use('Agg')

import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]

myexplode = [0.2, 0, 0, 0]

mycolors = ["black", "hotpink", "b", "#4CAF50"]

plt.pie(y, labels = mylabels , colors = mycolors, explode = myexplode)
plt.legend(title = "Four Fruits:",loc='lower left')
plt.show()

#Two  lines to make our compiler able to draw:
plt.savefig(sys.stdout.buffer)
sys.stdout.flush()
```



Box Plot

- A box plot, also known **as a box-and-whisker plot**, provides a visual summary of the distribution of a dataset.
- It represents **key statistical measures** such as the median, quartiles and potential outliers in a concise and intuitive manner. It displays the summary statistics of a dataset, including the minimum, first quartile (Q1), median (Q2), third quartile (Q3), and maximum.

- **Box plots** are particularly **useful for comparing distributions** across different groups or identifying anomalies in the data.
- The **box represents the interquartile range (IQR)** between the first and third quartiles, while **whiskers extend** from the box to the minimum and maximum values.
- **Outliers**, if present, may be displayed as individual points beyond the whiskers.

- (Quartiles in Python are statistical measures that divide a dataset into four equal parts, each containing 25% of the data. They are typically calculated using libraries like NumPy and Pandas, which provide functions to easily find these values.)

- **Imagine** you have the exam scores of students from three classes.
- A box plot is a way to show how these scores are spread out −

- **Minimum and Maximum** − The smallest and largest scores are shown as the **ends of the plot.**

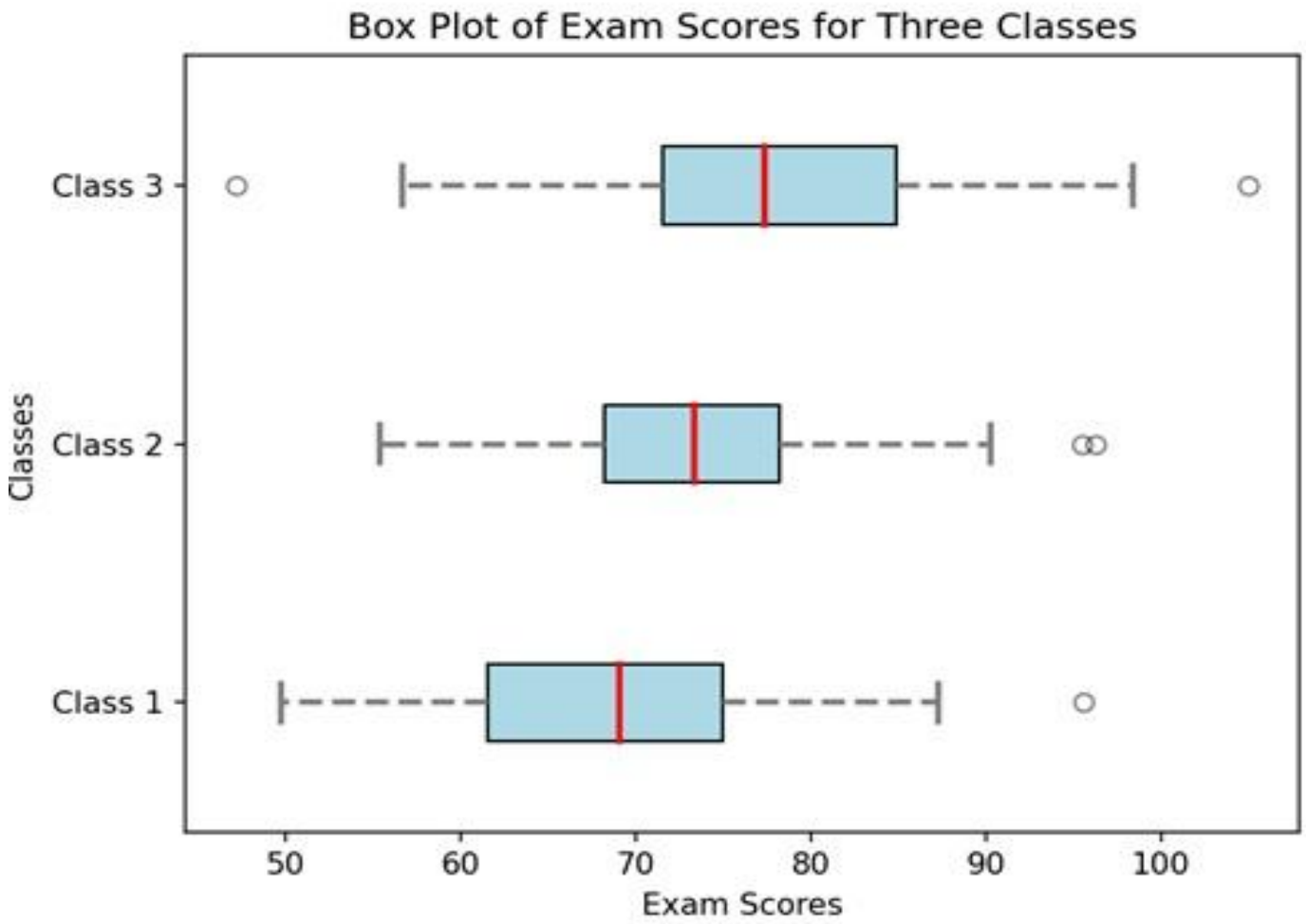
- **Quartiles (Q1, Q2, Q3)** − The scores are split into four parts. The **middle score is the median (Q2).** The scores **below the median are the first quartile (Q1),** and those above are the third quartile (Q3). It **helps** you see where most of the scores lie.

- **Interquartile Range (IQR)** − The **range between Q1 and Q3** is called the interquartile range.

- **Box** − The **box in the middle represents the interquartile range**. So, it is showing you where half of the scores are.

- **Whiskers − Lines (whiskers) extend from the box to the smallest and largest scores,** helping you see how spread out the scores are.

- **Outliers** − If there are **any scores way above or below the rest,** they might be shown as dots beyond the whiskers. These are like the standout scores.


Box Plot of Exam Scores for Three Classes
Class 3
Class 2
Class 1
Classes
50
60
70
80
90
100
Exam Scores

- We can create a **box plot in Matplotlib using the boxplot() function.**

- This **function allows us to customize the appearance** of the box plot, such as changing the whisker length,
  - adding notches, and
  - specifying the display of outliers.

The boxplot() Function

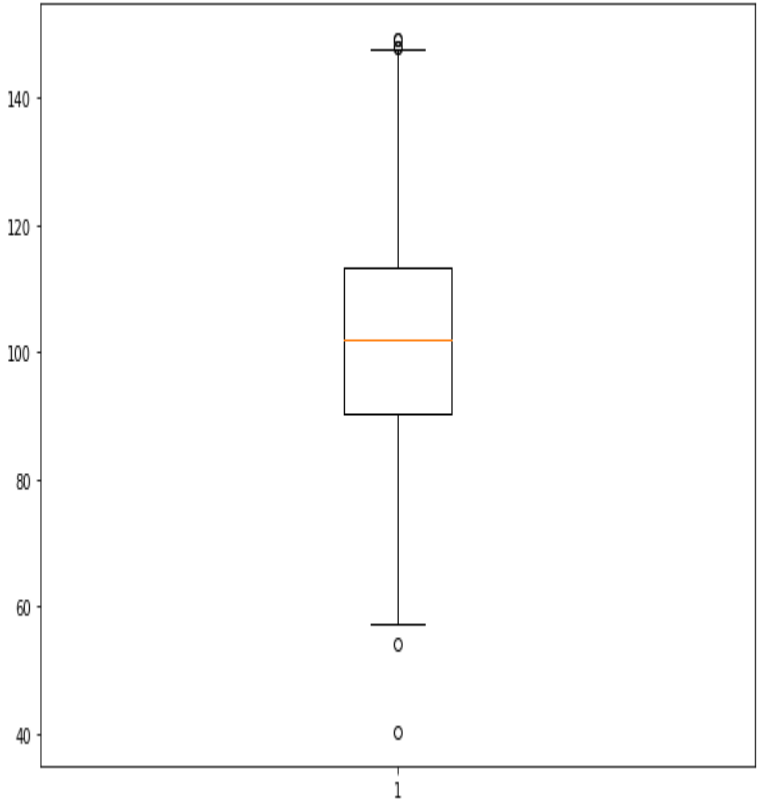
- The **boxplot() function in Matplotlib** takes one or more datasets as input and generates a box plot for each dataset.

- **Syntax**
  - plt.boxplot(x, notch=None, patch_artist=None, widths=None, labels=None, ...)

Where,

- **x** is the dataset or a list of datasets for which the box plot is to be created.
- If **notch (optional)** is True, it creates a vertical box plot; if False, creates a horizontal box plot.
- If **patch_artist (optional)** is True, it fills the box with color.
- **widths (optional)** represents the width of the boxes.
- **labels (optional)** sets **labels for each dataset**, useful when plotting multiple box plots.

```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(10)
data = np.random.normal(100, 20, 200)

fig = plt.figure(figsize =(10, 7))

plt.boxplot(data)
plt.show()
```



- **Explanation**
- **np.random.seed(10)**:
- Sets the "**seed" for the random number generator to a fixed value** (in this case, 10).
- This ensures that the ***same*** **sequence of random numbers is generated** every time the script is run, making the code reproducible.

- **data = np.random.normal(100, 20, 200)**:
- Generates an array of random numbers following a normal (Gaussian) distribution:
  - 100: The mean (center) of the distribution.
  - 20: The standard deviation (spread) of the distribution.
  - 200: The number of data points to generate.
- **The result is stored in the variable data**.

- **fig = plt.figure(figsize =(10, 7))**:
- Creates a new figure (the entire window or page the plot is drawn on) and sets its **size to 10 inches wide by 7 inches tall.**
- The **resulting figure object is stored in the variable fig.**

- **plt.boxplot(data)**:
- Generates a box plot (or box-and-whisker plot) from the data stored in the data array.
- This type of plot displays the **distribution of numerical data and skewness** through the display of the five-number summary: minimum, first quartile (Q1), median (Q2), third quartile (Q3), and maximum.

Error Plot

- Error plots display the **variability or uncertainty associated with each data point in a dataset.**

- They are **commonly used in**
  - scientific research,
  - engineering and
  - statistical analysis to visualize measurement errors, confidence intervals, standard deviations or other statistical properties of the data.

- By **incorporating error bars into plots**, we can convey not only the central tendency of the data but also the **range of possible values around each point.**

- **(**A **confidence interval (CI)** in Python is a **calculated range of values** that is likely to contain an unknown population parameter (like the mean or proportion), based on sample data.
- Libraries like scipy.stats and statsmodels provide functions to easily compute these intervals. **)**

- **The errorbar() Function**

- The errorbar() function in Matplotlib takes x and y coordinates, indicating the uncertainty in data points.

- **This function is** particularly useful for visualizing the range of possible values around each data point, providing a **better understanding of how certain or uncertain the measurements are.**

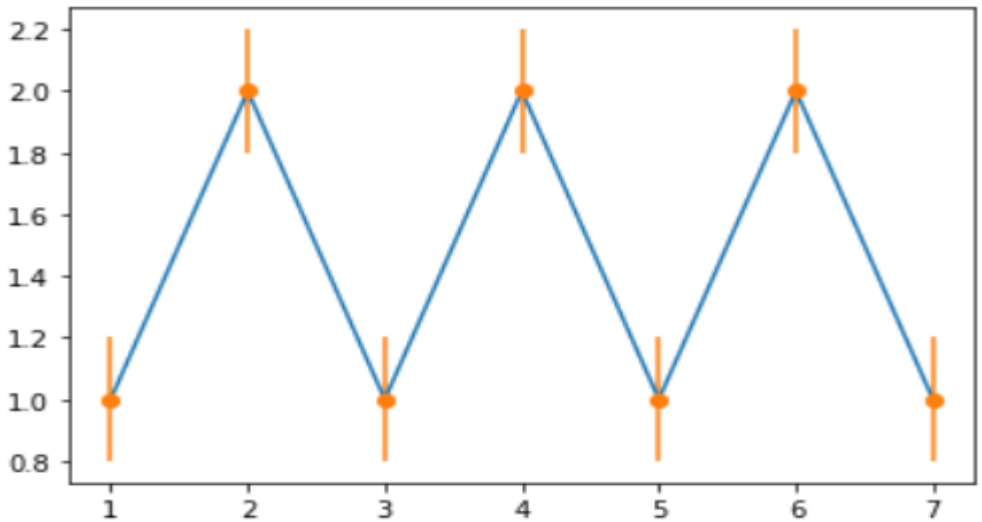
- **Following is the syntax of the errorbar() function in Matplotlib −**
- matplotlib.pyplot.errorbar(x, y, yerr=None, xerr=None, fmt='', barsabove=False, lolims=False, uplims=False, xlolims=False, xuplims=False, errorevery=1, capthick=None, *, data=None, **kwargs)
- Where,

- **x** is the x-coordinates of the data points.
- **y** is the y-coordinates of the data points.
- **yerr** is the error in the y-direction. It can be a **scalar or an array.**
- **xerr** is the error in the x-direction. It can be a **scalar or an array.**
- **fmt** is the format code controlling the **appearance of lines and markers.**
- If **barsabove** is True, error bars are **drawn above data points.**
- **lolims, uplims, xlolims, xuplims** are the Booleans indicating whether the errors represent lower/upper limits or if x-limits are specified.
- **errorevery** subsample the error bars by an integer factor.
- **capthick** is the thickness of the caps on the error bars.

- import matplotlib.pyplot as plt
- x =[1, 2, 3, 4, 5, 6, 7]
- y =[1, 2, 1, 2, 1, 2, 1]

- **y_error = 0.2**

- plt.plot(x, y)



- **plt.errorbar(x, y,yerr = y_error,fmt ='o')**

- **Explanation**
- x = [1, 2, 3, 4, 5, 6, 7]–
  - Defines a list named **x containing the data points** for the x-axis.

- y = [1, 2, 1, 2, 1, 2, 1]
  - Defines a list named **y containing the data points** for the y-axis.

- y_error = 0.2
  - Defines a **variable y_error with a single scalar value**.
  - This value represents the constant vertical error margin for all data points [1].

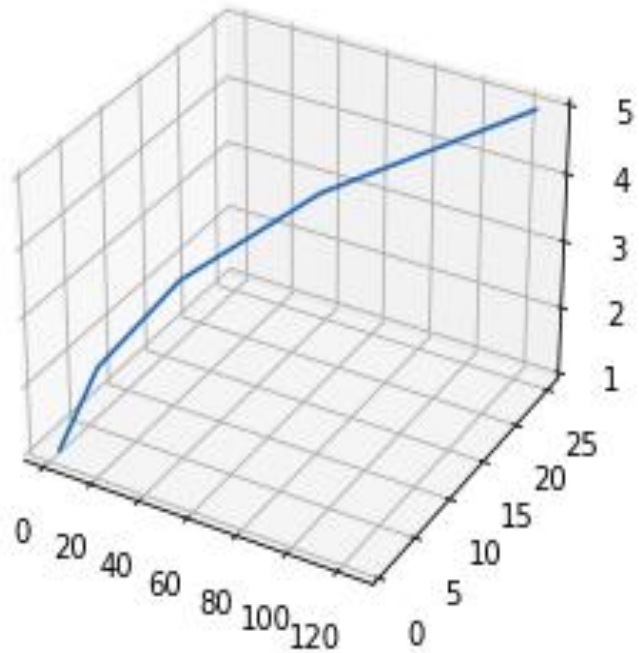
- plt.plot(x, y)
  - Generates a **line plot using the x and y data**.
  - By default, this will draw solid lines connecting the data points

- plt.errorbar(x, y, yerr=y_error, fmt='o’)
  - Overlays an error bar plot on the same figure.
  - * x, y: Specifies the **center coordinates for the error bars**.
  - * yerr=y_error: Specifies the vertical **error amount (±0.2) for each point.**
  - * fmt='o': Specifies the format **string to use circles ('o')** as markers for the data points, instead of the default line

3D Plots in Matplotlib

- Sometimes, **data visualization requires a three-dimensional perspective.**
- We'll delve into creating 3D plots to visualize **complex relationships and structures** within multidimensional datasets.

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [1, 4, 9, 16, 25]
z = [1, 8, 27, 64, 125]

fig = plt.figure()

ax = plt.axes(projection = '3d')
ax.plot3D(z, y, x)

plt.show()
```



- The above code lets the **creation of a 3D plot in Matplotlib.**
- different types of 3D plots can be created like scatter plots, contour plots, surface plots, etc.

- Explanantion
- x = [1, 2, 3, 4, 5]
- y = [1, 4, 9, 16, 25]
- z = [1, 8, 27, 64, 125]
- Defines three lists representing coordinates in 3D space.
- x values are linear (1-5).
- y values are squares (x^2).
- z values are cubes (x^3).
- These will map to (z_i, y_i, x_i) for each point i.

- **plt.figure():**
  - Creates a new figure (the canvas).

- **plt.axes(projection='3d’):**
  - Adds a 3D axes object to the figure.
  - The projection='3d' parameter is crucial as it enables 3D plotting capabilities, allowing for depth in the graph

- **x.plot3D(z, y, x)**
  - Uses the plot3D function to create a 3D line plot using the data defined earlier.
  - The data is passed in order of axis: (Z-axis, Y-axis, X-axis)

- **plt.show()**
- Renders the plot window, allowing you to view and interact with the 3D graph (e.g., rotate it to view from different angles)

Scikit-learn

- Scikit-learn is **an open-source Python library that** simplifies the process of building machine learning models.

- It offers a **clean and consistent interface** that helps both **beginners and experienced users work efficiently.**
  - Supports tasks like classification, regression, clustering and preprocessing
  - Makes **model building fast and reliable**
  - Provides ready-to-use tools for training and evaluation
  - **Reduces complexity** by avoiding **manual implementation of algorithms**

Scikit-learn simplifies machine learning with pre-built tools and algorithms.

Installing and Using Scikit-learn

- **Installing and Using Scikit-learn**
- Before we **start building models** we need to install **Scikit-learn.**
- It requires Python 3.8 or newer and **depends** on two important **libraries: NumPy and SciPy.**

- Make sure these are installed first.

- To install Scikit-learn run the following command:
  - **pip install -U scikit-learn**

- This will download and install the latest version of Scikit-learn along with its dependencies.

Features of Scikit-learn

- Scikit-learn is used because it makes **building machine learning models straightforward and efficient.**

- Here are some important reasons:
- **Ready-to-Use Tools**:
- It provides **built-in functions for common tasks** like data preprocessing, training models and making predictions.
- **This *saves time*** by avoiding the need to code algorithms from scratch.

- **Easy Model Evaluation**:
- With tools like **cross-validation and performance metrics** it helps to measure how well our model works and identify areas for improvement.
- **(Cross-validation is a statistical technique** for **assessing how well a model generalizes** to an **independent, unseen dataset**.
- It is **crucial for detecting and preventing** issues **like overfitting or underfitting.)**

- **Wide Algorithm Support**:
- It offers many **popular machine learning algorithms** including **classification, regression and clustering** which gives us flexibility to choose the right model for our problem.

- **Smooth Integration**:
- **Built on top of important Python libraries like NumPy and SciPy** so it fits into our existing data analysis workflow.

- **Simple and Consistent Interface**:
- The same straightforward **syntax works across different models** helps in making it easier to learn and switch between algorithms.

- **Model Tuning Made Easy**:
- Tools like grid search help us **fine-tune our model's settings to improve accuracy without extra hassle.**

Benefits of using Scikit-learn

- **User-Friendly**: Scikit-learn's **consistent and simple interface** makes it **accessible for beginners and best for experts.**

- **Time-Saving: Pre-built tools and algorithms** reduce development time which allows us to focus more on solving problems than coding details.

- **Better Model Performance**: **Easy-to-use tuning and evaluation tools** helps in improving model accuracy and reliability.

- **Flexible and Scalable**: Supports a wide range **of algorithms and integrates smoothly with other Python libraries** helps in making it suitable for projects of any size.

- **Strong Community Support**: A large, active community **ensures regular updates, extensive documentation and plenty of resources** to help when we get stuck.

Thank you